

Intelligent systems for industry, supply chain and environment

LESSON 16

Using new models in your practice

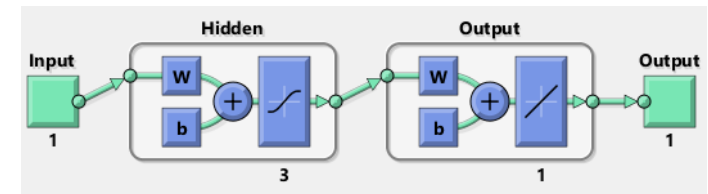
Linear regressors and classifiers (with coding)

Feed Forward Neural and Overfitting!



Outline

- Agents Memory and Context management
- Loading, EDA/plotting in Matlab
- New model?
 - What to know
 - What to do
- The Bias/Variance Tradeoff (Dilemma)
- Creation and use (with code) of the first complete machine learning models
 - Linear regressors
 - Linear classifiers
 - First neural networks
 - Overfitting!!
- Main points



Agents for ML

Where agents can be useful, and what they can do

ML workflow phase	What the agent can help with	What must be validated by humans
Problem definition	reconstruct context, extract constraints	task definition, target variable
Data	inspect files, detect missing values, propose cleaning	leakage, units, labels
Representation	suggest features, encode categorical variables	feature meaning, causality risk
Model	propose baselines and candidates	model choice and assumptions
Evaluation	compute metrics, compare models	test split validity, statistical meaning
Reporting	generate tables, plots, explanations	claims and conclusions
Monitoring	draft drift checks and dashboards	deployment risk and thresholds

Memory of Agents

Context management

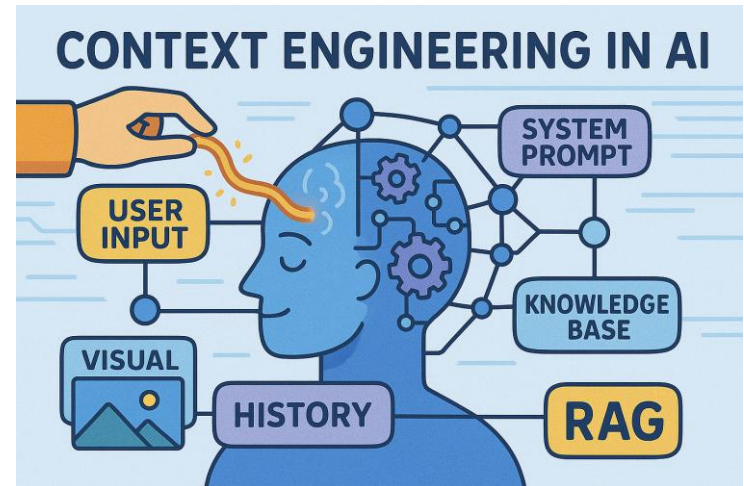
Control, manage, compress, restart...



Context in LLMs and agents

In LLMs and agents, context is the active information state passed to the model at inference time, including

- user instructions,
- conversation history,
- retrieved documents,
- tool results,
- system rules,
- memory summaries,
- available tool definitions.

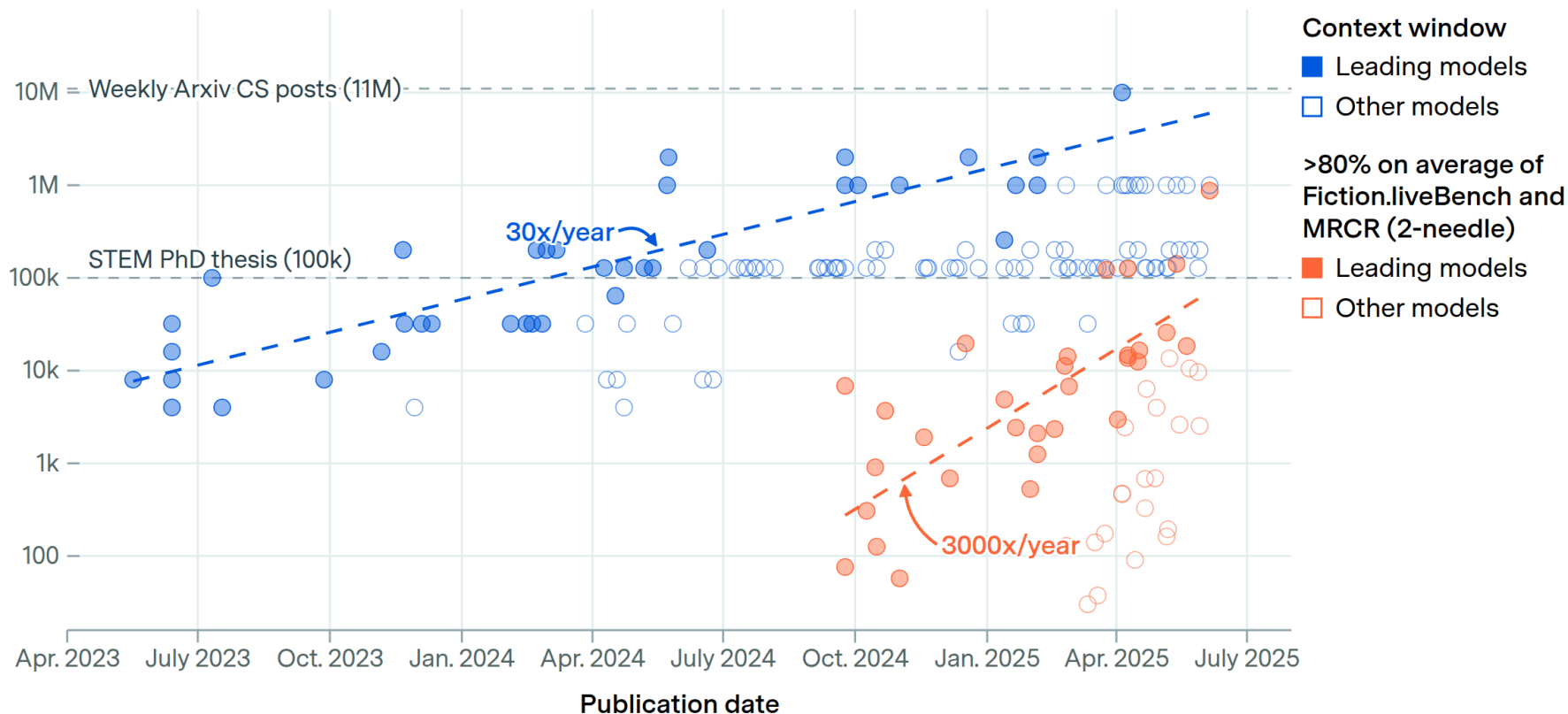


Context size is increasing!

We have more space to say something meaningful.

Frontier LLMs accept increasingly long inputs, and their usable context length has grown even faster

Tokens



Context size matters

Haystack: 500,000 tokens of unrelated documents.
Needle: "The secret project code is ALPHA-73."
Question: "What is the secret project code?"
Expected answer: "ALPHA-73."

As of mid-2026, a 1-million-token context window is indeed the state-of-the-art (SoA) standard for top-tier, large-scale cloud AI models, enabling the processing of immense datasets, such as entire codebases or hundreds of documents, in a single prompt!

- A 1M token window is equivalent to about 750,000 words, or over 2,000 pages of text
- "Needle in a haystack" tests show high accuracy. However, some users report increased latency and potential performance degradation at the 1M limit

Context size (local and cloud)

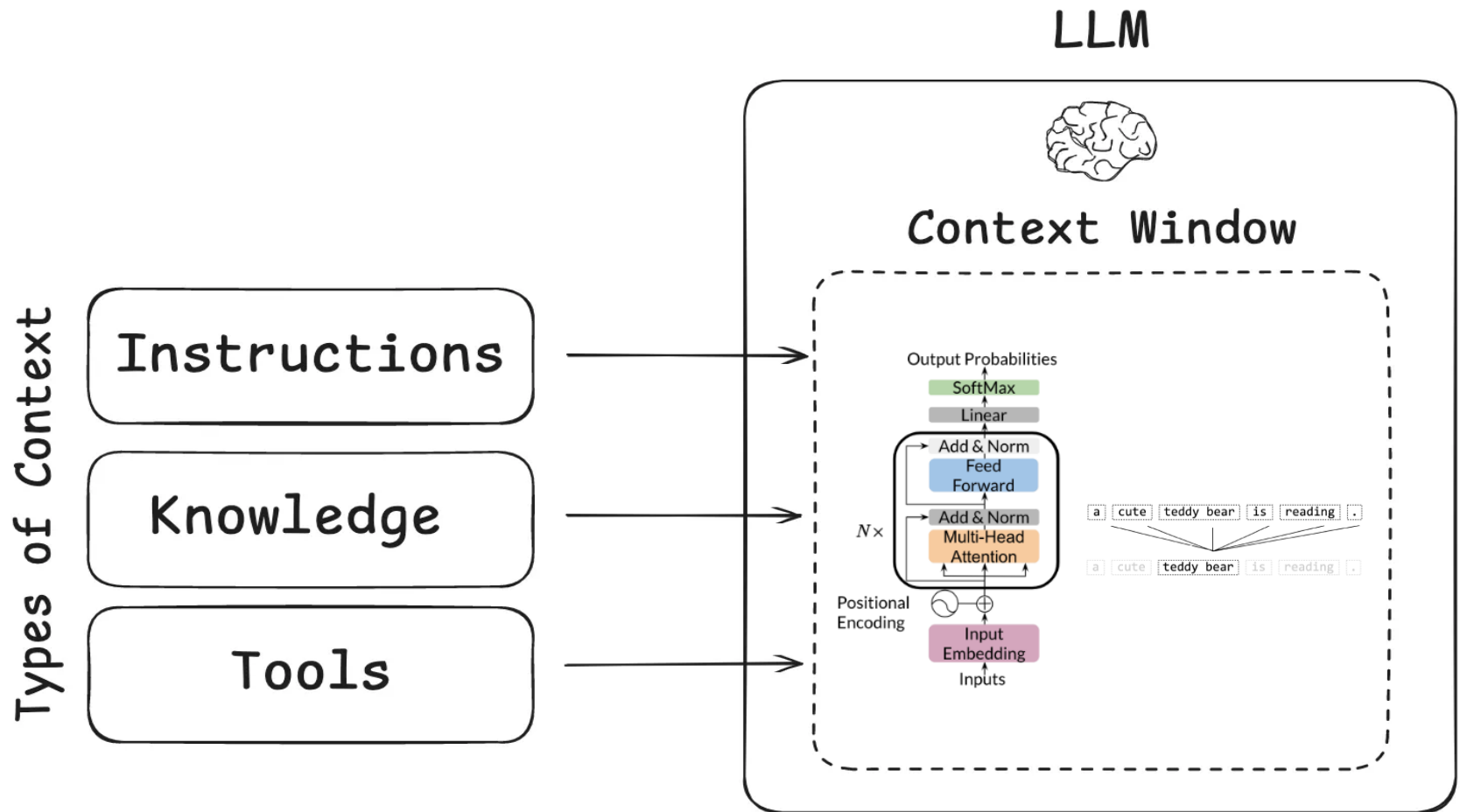
LLM Context: Bigger Window $\neq$ Better Understanding

Dimension	Cloud frontier models	Local / open-weight models
Typical context	200K–1M tokens on selected recent models	8K–128K practical, depending on RAM/VRAM
Example models	Claude Opus / Sonnet: up to 1M; Haiku: ~200K	Qwen2.5-Coder 7B, GPT-OSS 20B, Qwen3-Coder 30B
Main advantage	Large context, robust tool use, fast infrastructure	Privacy, data control, low marginal cost
Main limitation	Data leaves the local machine; cost grows with tokens	Context is limited by hardware and speed
Hidden risk	More context can still cause context rot	Tool-use may fail before context size becomes useful

LLM context is rebuilt at every chat turn

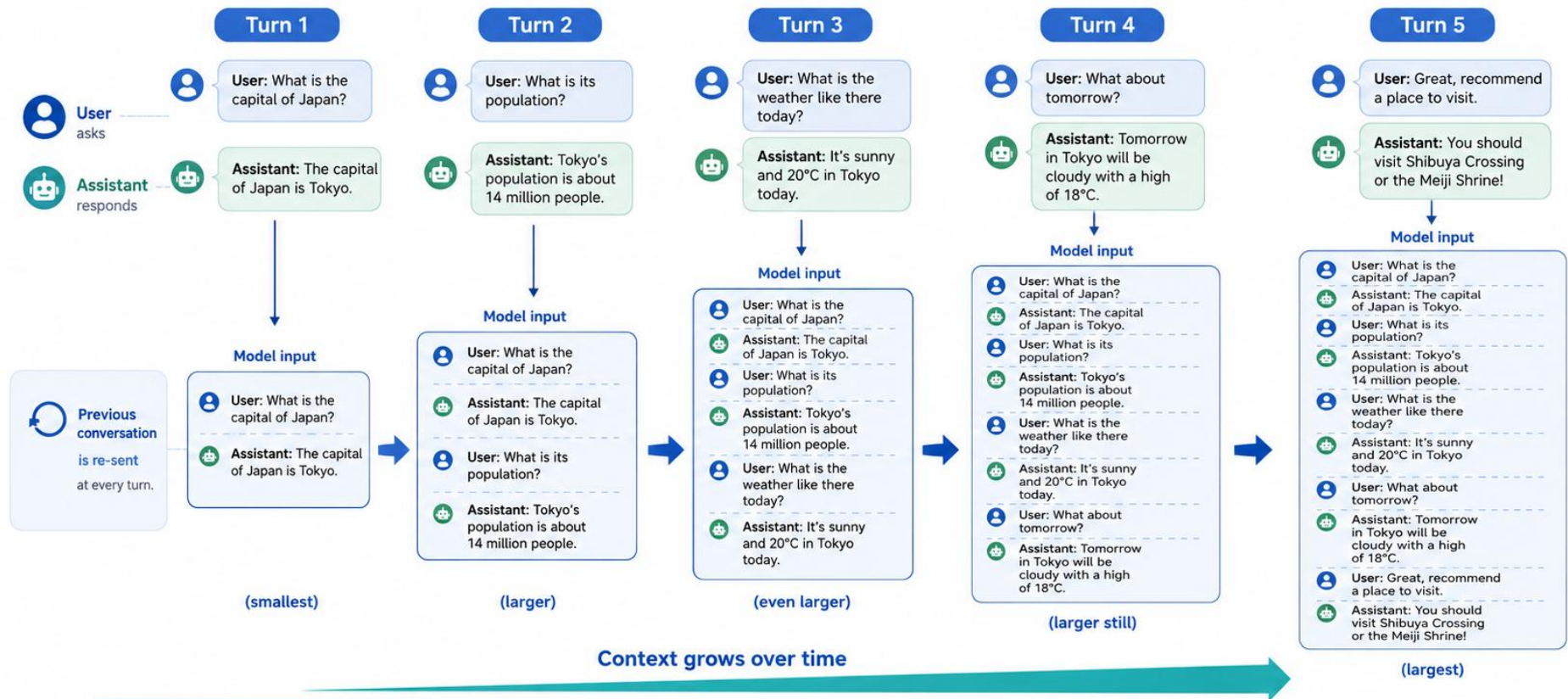
- An LLM call is **stateless**.
After the model generates an answer, its internal active context is discarded.
- What persists is not inside the LLM, but in the surrounding application, such as ChatGPT, Claude Code, Codex, Cursor, or a custom agent framework.

LLM context is rebuilt at every chat turn



Context grows.... in chats and w/ agents

The full prior conversation is sent back as input at each new turn.



Example

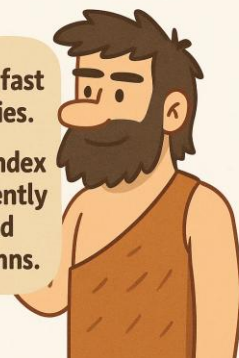
Assume
system prompt = 200 tokens,
each user message $\approx$ 50 tokens
each assistant response $\approx$ 300 tokens.

- Turn 1: $|\text{context}_1| = 200 + 50 = \mathbf{250 \text{ tokens}}$ sent to LLM.
Response: **300** tokens. Log size after: 350.
- Turn 2: $|\text{context}_2| = 200 + 50 + \mathbf{300} + 50 = \mathbf{600 \text{ tokens}}$ sent.
Response: **300**. Log size after: 700.
- Turn 3: $|\text{context}_3| = 200 + 50 + \mathbf{300} + 50 + \mathbf{300} + 50 = \mathbf{950 \text{ tokens}}$ sent.
- Turn N: $|\text{context}_n| \approx 200 + N \cdot 50 + (N-1) \cdot 300 \approx \underline{350 \cdot N \text{ tokens}}$.

CAVEMAN COMPRESSION

In order to optimize the database query performance, we should consider implementing an index on the frequently accessed columns.

Need fast queries.
Add index frequently used columns.



CAVEMAN SKILL



+



+



REDUCE AI TOKENS
BY 75%

LLM context is rebuilt at every chat turn

At every new chat turn, the application builds a new **active context** by selecting information from different external sources:

- current user message
- recent conversation history
- system and developer instructions
- relevant summaries
- saved memories
- project files
- retrieved documents
- tool results
- selected logs or code snippets

Context management techniques

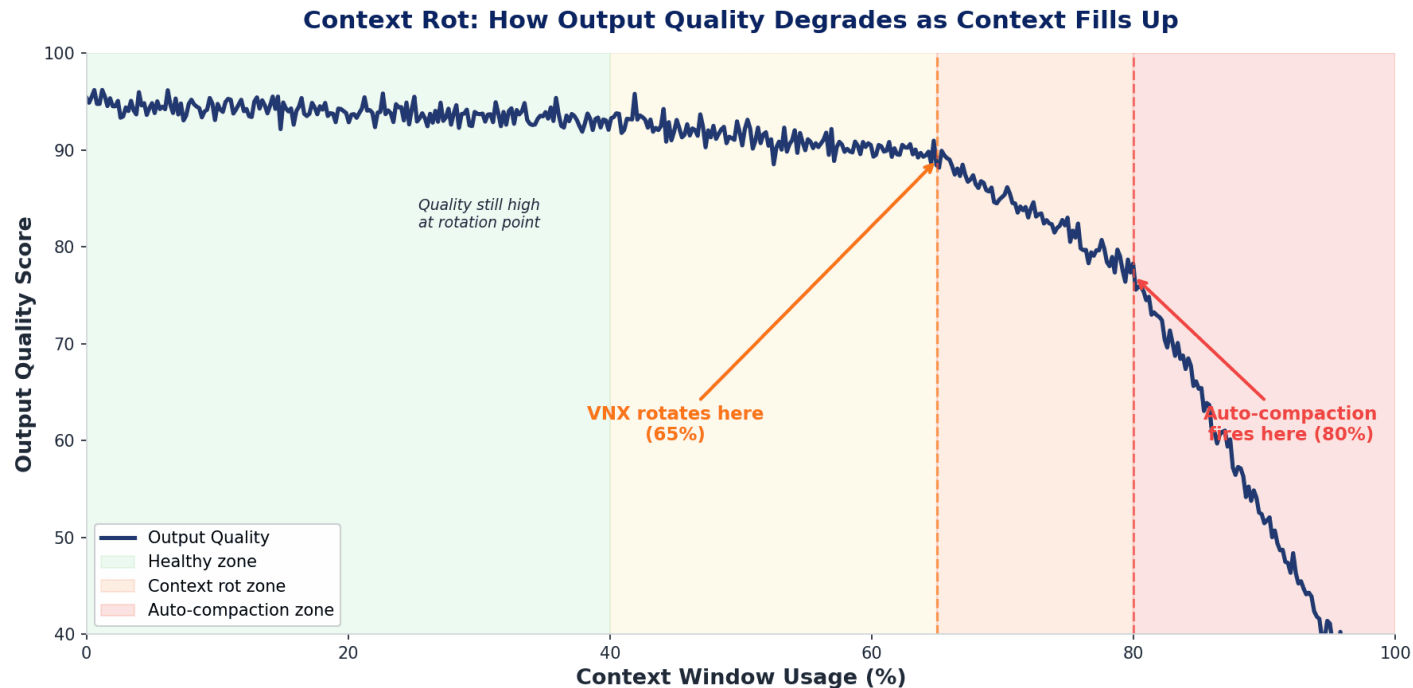
Context engineering is a set of mechanisms to prevent the context rot obtained by managing what the model can access.

Main methods are:

- Offloading
- RAG
- Tool loadout / Dynamic tool selection
- Pruning
- Summarization
- Quarantine

Context rot

Never use the full context from the beginning



- The context window should not be treated as a hard storage limit only.
- As the conversation grows, output quality may degrade before the context is technically full: relevant information becomes harder to prioritize, instructions can compete with older content, and the model may start ignoring or misusing evidence.
- Practical systems therefore need context management strategies such as retrieval, summarization, rotation, and auto-compaction before quality collapses.

Context management 1

Offloading

Context Offloading is the act of storing information outside the LLM's context, usually via a tool that stores and manages the data.

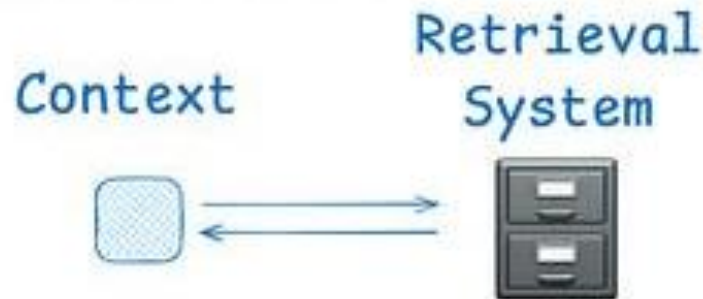


Store large or persistent information outside the active prompt — e.g., files, memory stores, scratchpads, databases — and let the agent retrieve only what is needed later.

Context management 2

RAG

Retrieval-Augmented Generation (RAG) is the act of selectively adding relevant information to help the LLM generate a better response.

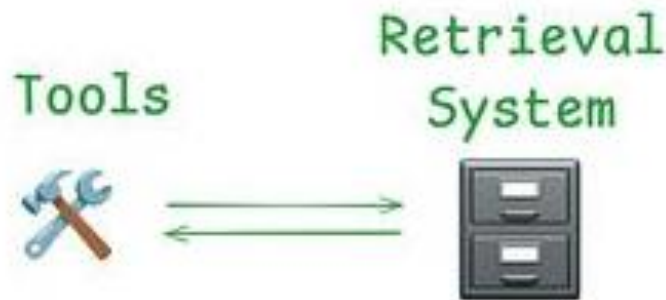


Retrieve relevant external knowledge at inference time instead of putting all documents into the prompt; useful for grounding, scalability, and reducing context overload.

Context management 3

Loadout /dynamic tool selection

Tool Loadout is the act of selecting only relevant tool definitions to add to your context.

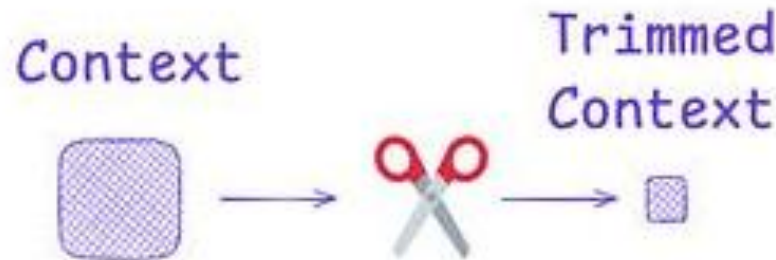


Expose only the tools relevant to the current task; too many tool definitions can confuse the model, increase tokens, and cause wrong tool calls.

Context management 4

Pruning

Context Pruning is the act of removing irrelevant or otherwise unneeded information from the context.

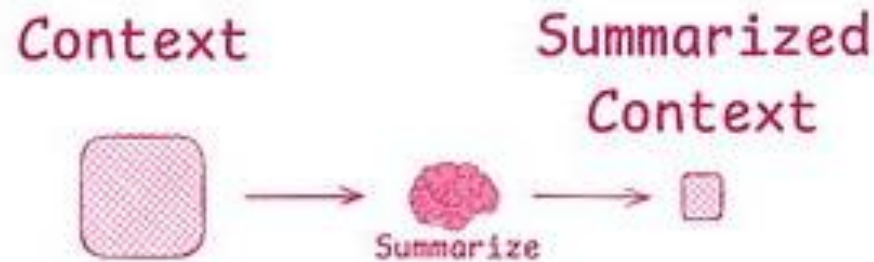


Remove irrelevant, obsolete, duplicated, or low-value context before the next model call; this reduces distraction and helps keep the active window focused.

Context management 5

Summarization

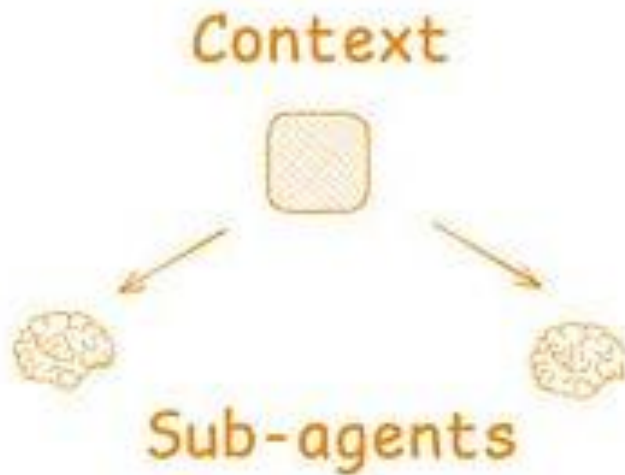
Context Summarization is the act of boiling down an accrued context into a condensed summary.



Compress older conversation or tool results into a compact working summary; useful, but risky if the summary loses decisions, constraints, numbers, or evidence.

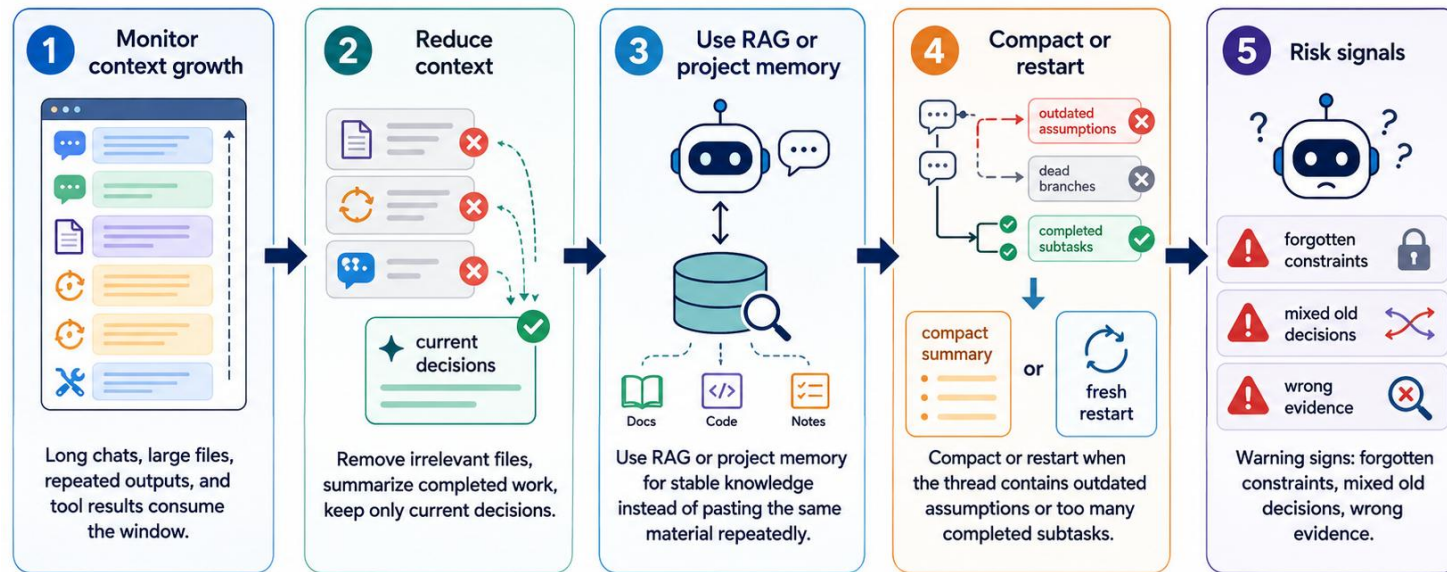
Context management 6

Context Quarantine is the act of isolating contexts in their own dedicated threads, each used separately by one or more LLMs.



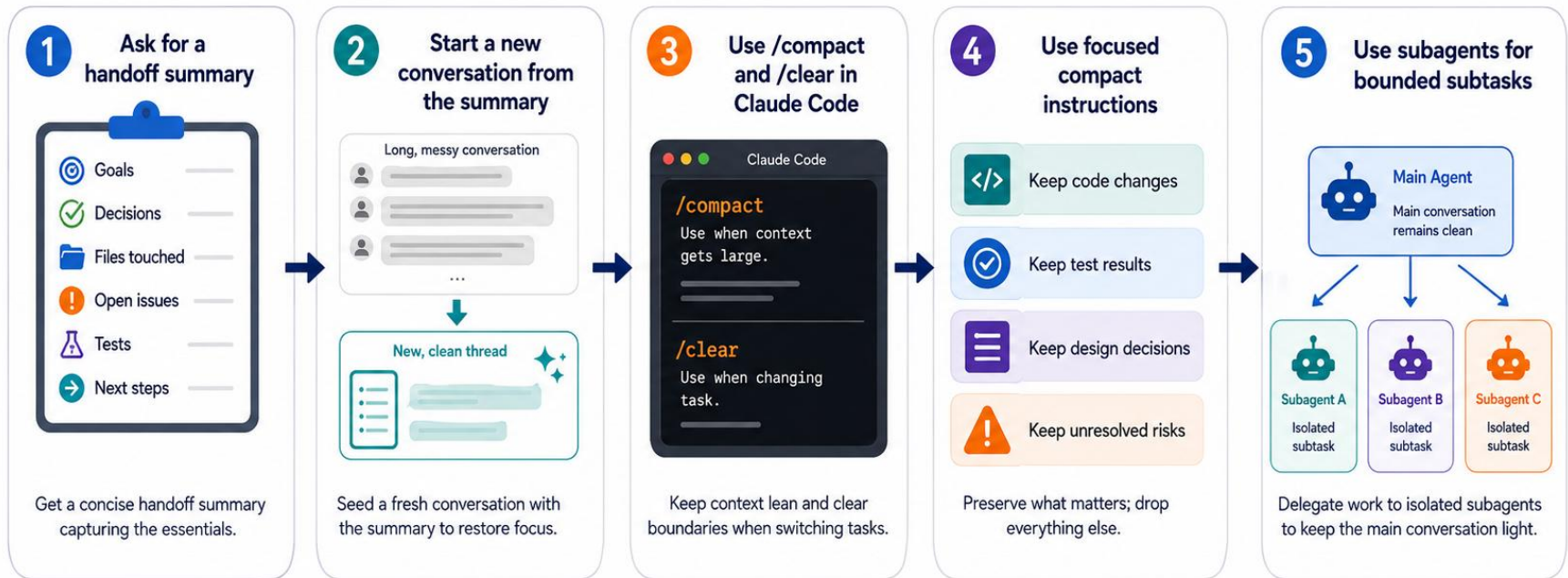
Isolate complex subtasks in separate threads or sub-agents so their noisy intermediate reasoning, tool outputs, and failures do not pollute the main agent context.

Managing context during an agentic workflow



- Monitor context growth: long chats, large files, repeated outputs and tool results consume the window.
- Reduce context by removing irrelevant files, summarizing completed work and keeping only current decisions.
- Use RAG or project memory for stable knowledge instead of pasting the same material repeatedly.
- Compact or restart when the conversation contains outdated assumptions, dead branches or too many completed subtasks.
- Risk signal: the agent starts forgetting constraints, mixing old decisions or reading the wrong evidence.

Operational practices for context hygiene



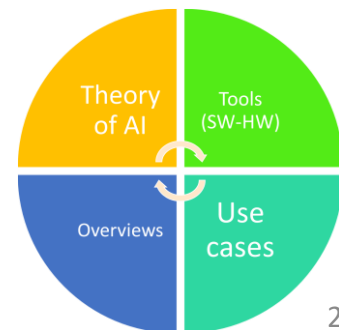
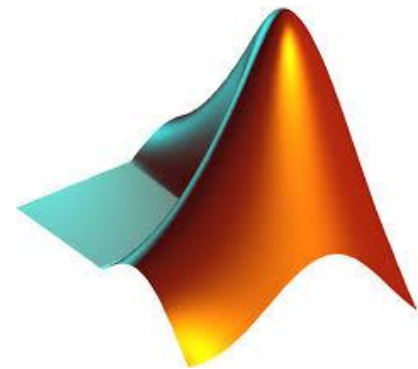
- Ask for a handoff summary: goals, decisions, files touched, open issues, tests, next steps.
- Start a new conversation from that summary when the previous one becomes too long or unfocused.
- In Claude Code, use `/compact` when context gets large and `/clear` when changing task.
- Use focused compact instructions, e.g. keep code changes, test results, design decisions and unresolved risks.
- Use subagents for bounded subtasks when separate context windows help preserve the main conversation.



Toolboxes

Classification errors

In python



Test the wine decision tree

```
# Define the actual wine cultivar names
```

```
wine_cultivars = ['Barolo', 'Grignolino', 'Barbera']
```

```
# 5. Create and visualize confusion matrix for training data
```

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
# Train a new decision tree with depth=3
```

```
clf_shallow = DecisionTreeClassifier(max_depth=3)
```

```
clf_shallow.fit(X, y)
```

```
# Get predictions on training data
```

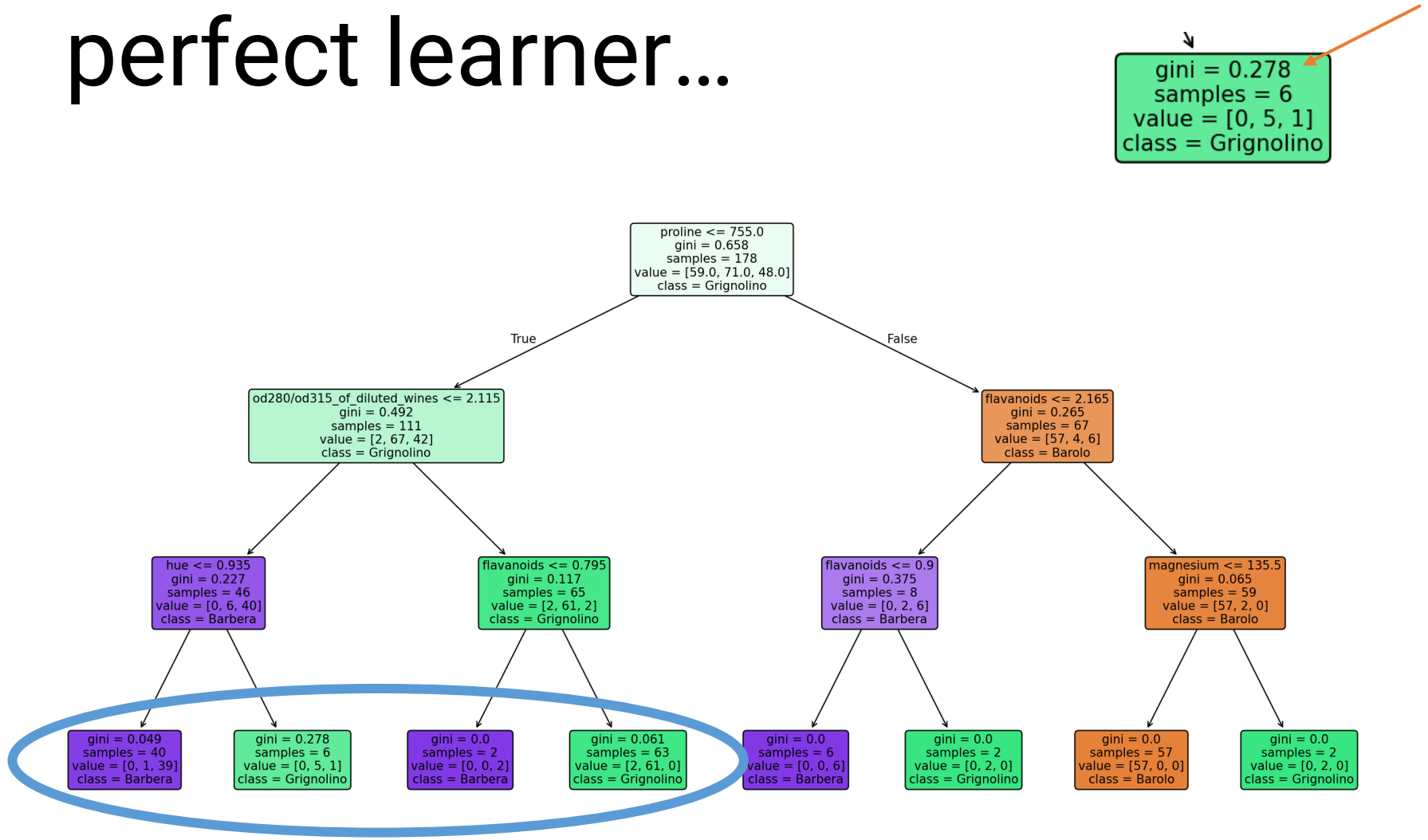
```
y_pred = clf_shallow.predict(X)
```

```
# Create and plot confusion matrix
```

```
cm = confusion_matrix(y, y_pred)
```

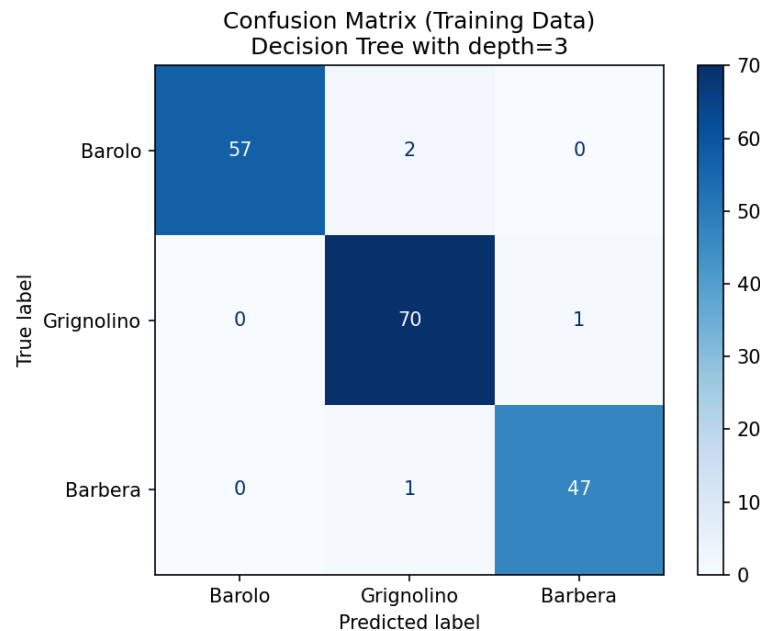
```
disp = ConfusionMatrixDisplay(confusion_matrix=cm,  
                               display_labels=wine_cultivars) # Using actual wine names
```

The shallow tree is not a perfect learner...



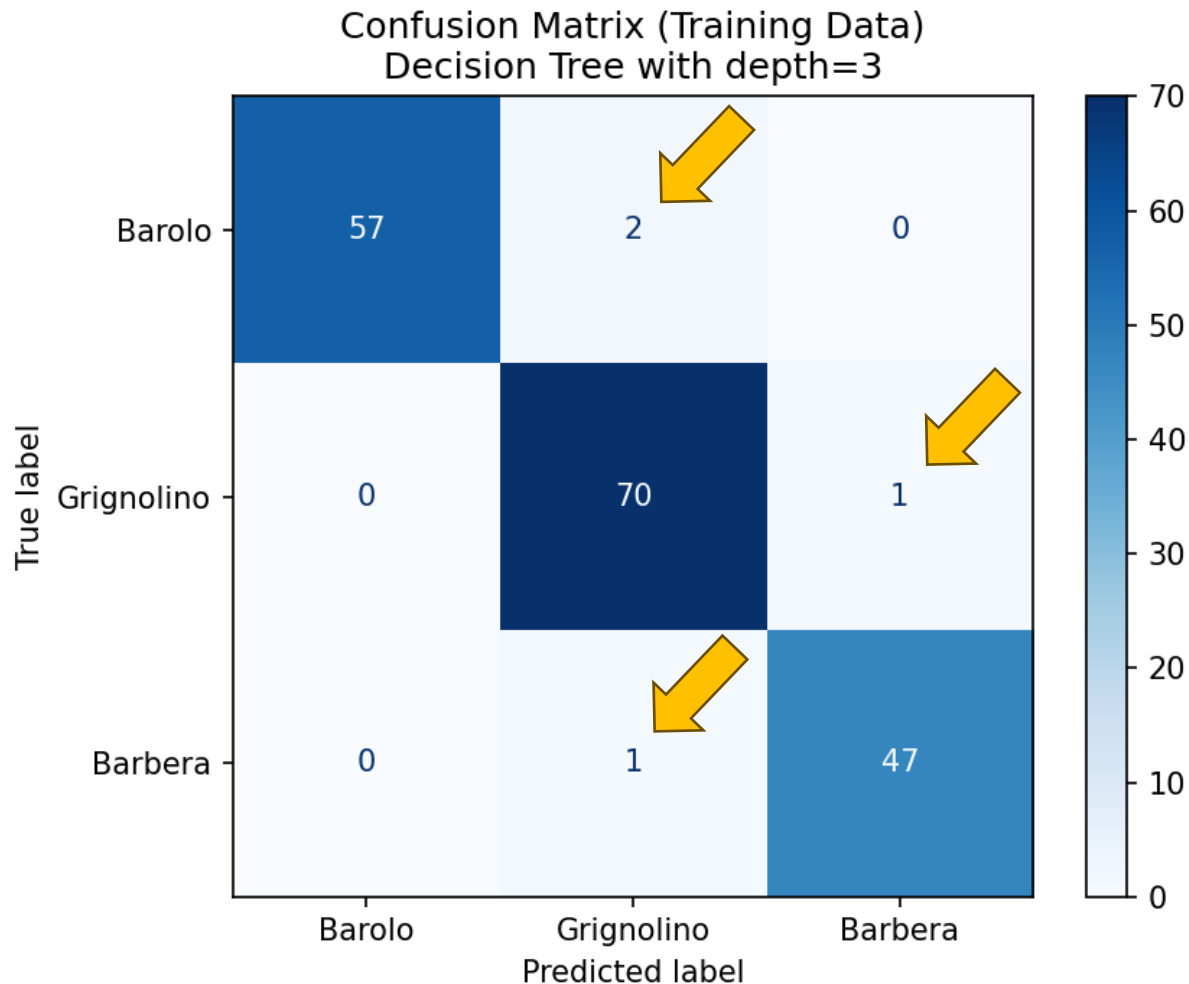
Let's check the accuracy

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm,  
                               display_labels=wine_cultivars) # Using actual wine names  
plt.figure(figsize=(10,10))  
disp.plot(cmap='Blues', values_format='d')  
plt.title('Confusion Matrix (Training Data)\nDecision Tree with depth=3')  
plt.show()
```

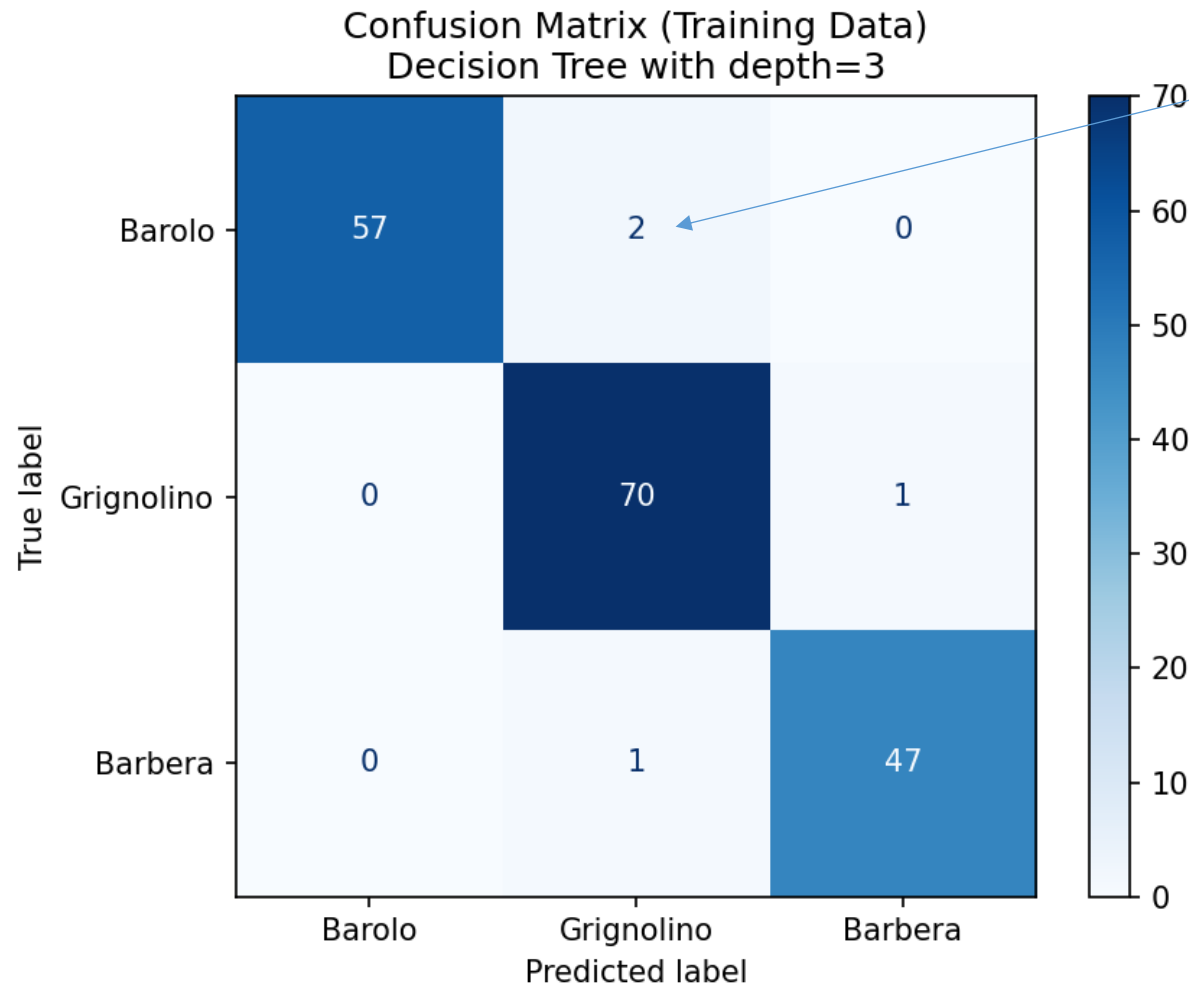


Let's check the accuracy

Where are the errors?



Let's check the accuracy



The tree output was "Grignolino" but the real cultivar was "Barolo"!!

What a unforgivable Mistake!!!

Process the accuracy

```
# Get predictions on training data
```

```
y_pred = clf_shallow.predict(X)
```

```
# Print accuracy score
```

```
accuracy = (y == y_pred).mean()
```

```
print("\nAccuracy on training data with depth=3:", round(accuracy * 100, 2), "%")
```

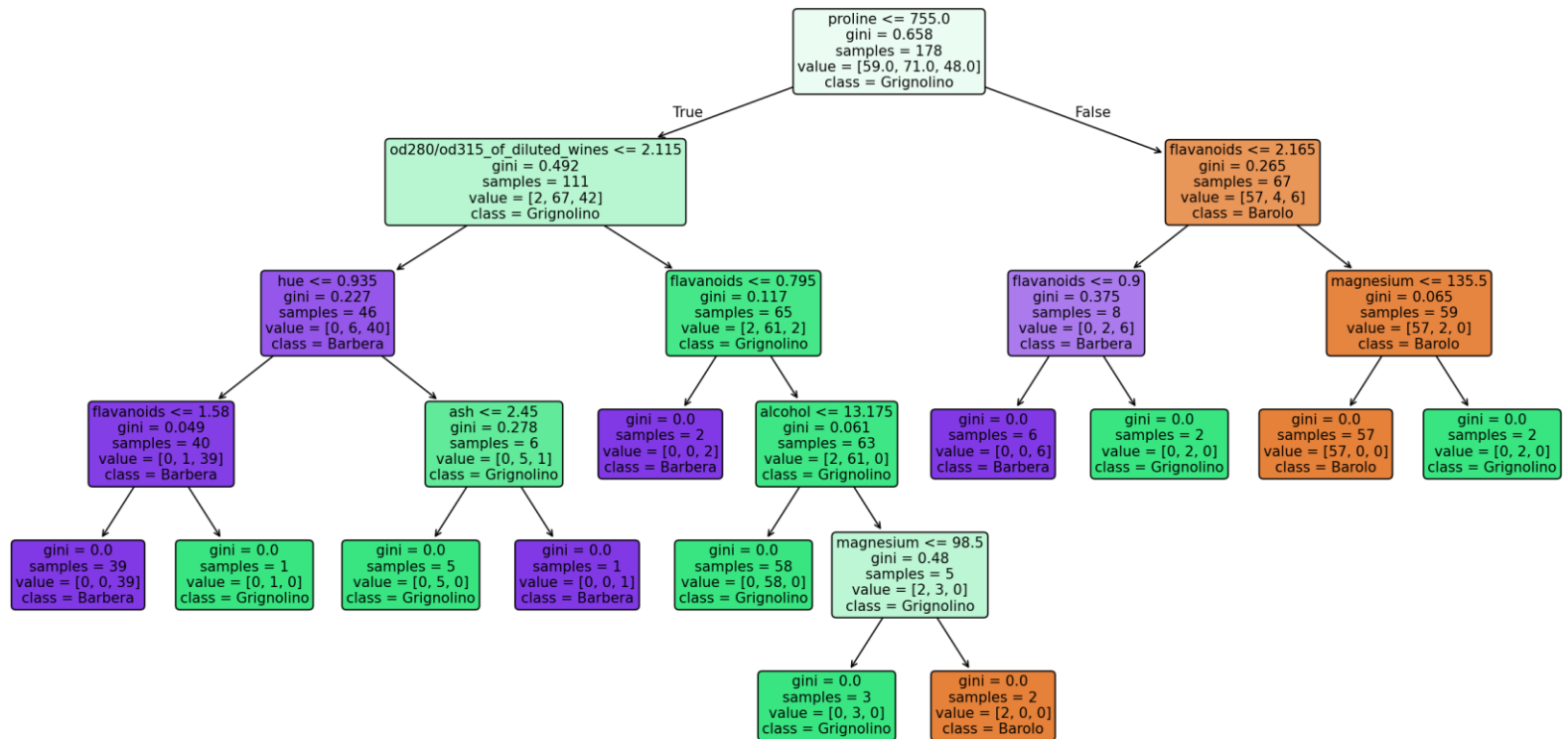
```
print("Note: Some misclassifications occur due to the limited tree depth")
```

```
Accuracy on training data with depth=3: 97.75 %
```

```
Note: Some misclassifications occur due to the limited tree depth
```

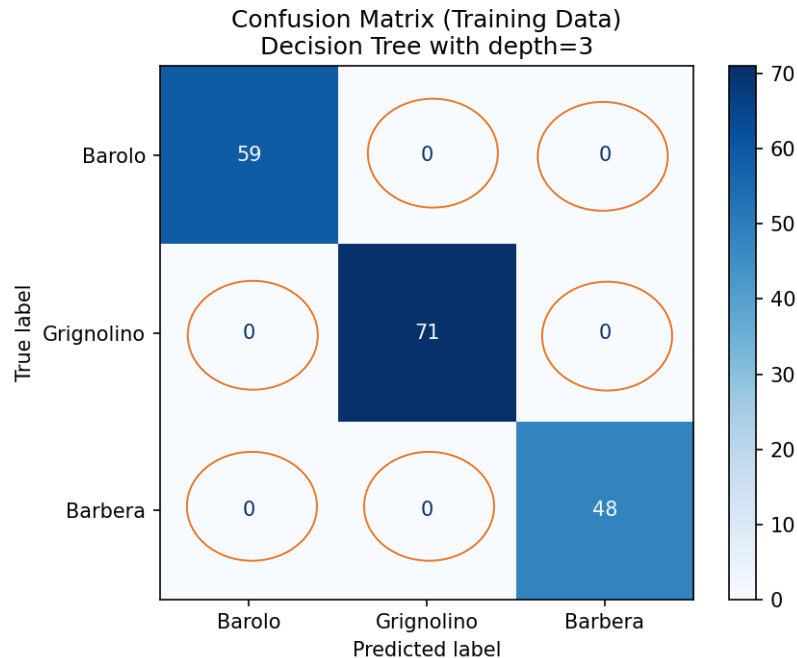

Adding layers

```
# Train a new decision tree with depth=3  
clf_shallow = DecisionTreeClassifier(max_depth=5)  
clf_shallow.fit(X, y)
```



We created a perfect learner

```
# Create and plot confusion matrix
cm = confusion_matrix(y, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=wine_cultivars) # Using actual wine names
plt.figure(figsize=(10,10))
disp.plot(cmap='Blues', values_format='d')
plt.title('Confusion Matrix (Training Data)\nDecision Tree with depth=3')
plt.show()
```



Process the accuracy

```
# Get predictions on training data
y_pred = clf_shallow.predict(X)

# Print accuracy score
accuracy = (y == y_pred).mean()
print("\nAccuracy on training data with depth=3:", round(accuracy * 100, 2), "%")
if accuracy == 1.0:
    print("We created a perfect learner!")
else:
    print("Note: Some misclassifications occur due to the limited tree depth")
```

```
Accuracy on training data with depth=5: 100.0 %
We created a perfect learner!
```

Is it a good idea to create a perfect learner?



- **Zero training error $\neq$ zero *generalization* error**
- **Memorisation & noise capture (overfitting)**
 - High-capacity models store spurious regularities and measurement noise, making predictions brittle when the distribution shifts (new vintages, sensor drift, etc.).
 - Overfitting is strongly suspected whenever zero training error.

Practical risks

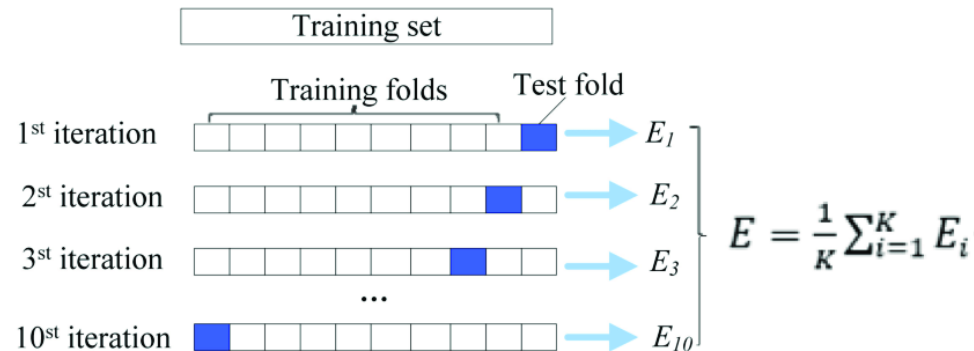
- **Unstable predictions:** small feature perturbations
---> different leaves/paths.
- **Poor maintainability:** model must be retrained frequently as data drift breaks memorised rules.
- **Misleading stakeholders:** an apparent 100 % metric hides real-world error.

We can test the generalization with the cross validation!

```
from sklearn import tree # the functions like tree.plot_tree()
from sklearn.model_selection import cross_val_score # Add this import
```

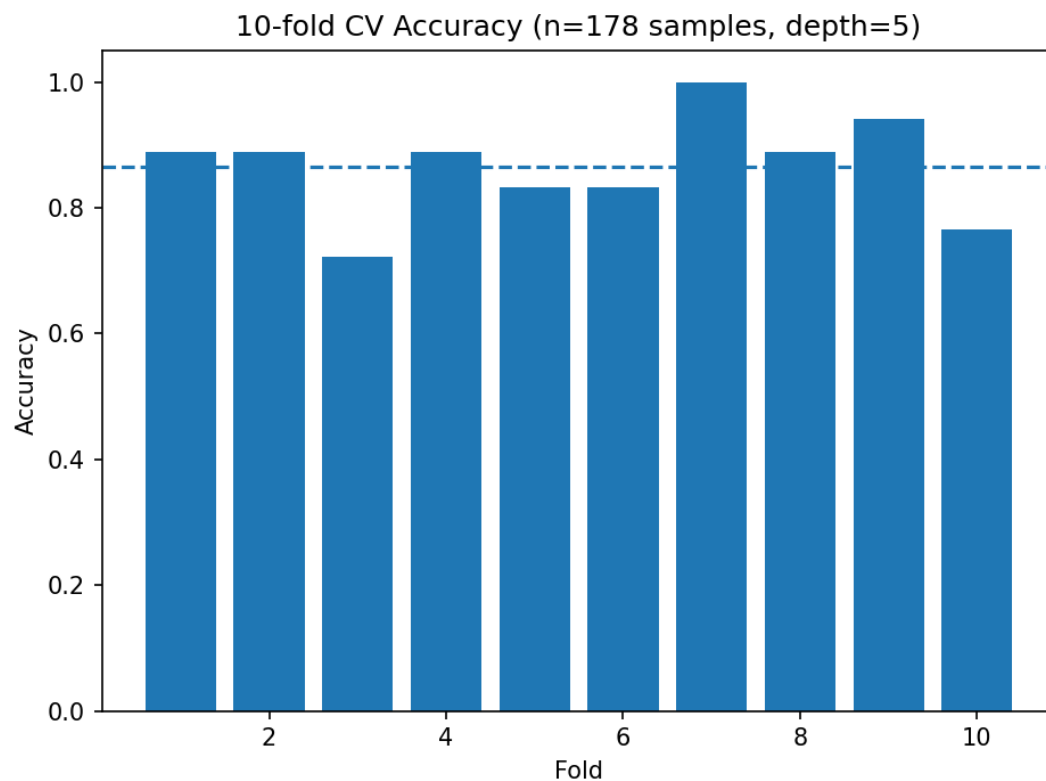
```
# 3. Evaluate
```

```
n_samples = len(X) # Add this line to define n_samples
train_accuracy = clf.score(X, y) # Training accuracy
cv_scores = cross_val_score(clf, X, y, cv=10) # 10-fold CV accuracies
cv_mean = cv_scores.mean()
```



4. Plot

```
plt.figure()
plt.bar(range(1, 11), cv_scores)           # one bar per fold
plt.axhline(cv_mean, linestyle='--')       # dashed line = mean accuracy
plt.xlabel("Fold")
plt.ylabel("Accuracy")
plt.title(f"10-fold CV Accuracy (n={n_samples} samples, depth={tree_depth})")
plt.tight_layout()
plt.show()
```

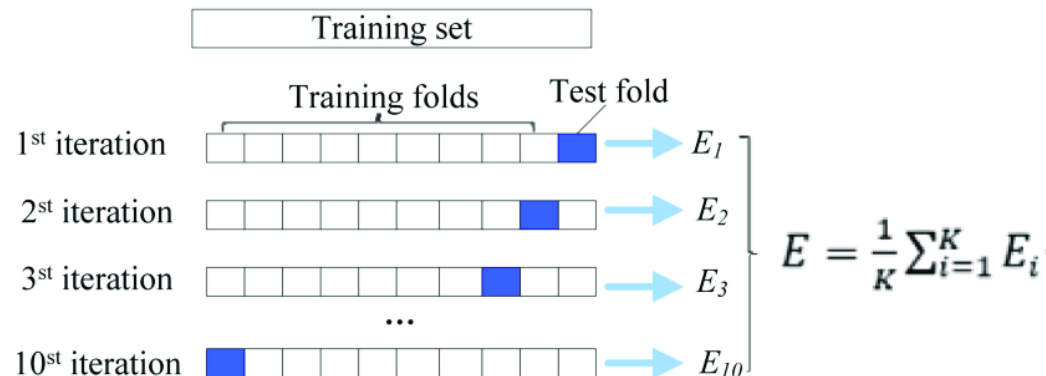


It's not so perfect!

Let's see the cross-validation accuracies....

```
# 5. (optional) print metrics
print("Training accuracy:", round(train_accuracy, 3))
print("Cross-validation accuracies:", np.round(cv_scores, 3))
print("Mean CV accuracy:", round(cv_mean, 3))
```

```
Training accuracy: 1.0
Cross-validation accuracies: [0.889 0.889 0.722 0.889 0.833 0.833 1.      0.889 0.941 0.765]
Mean CV accuracy: 0.865
```



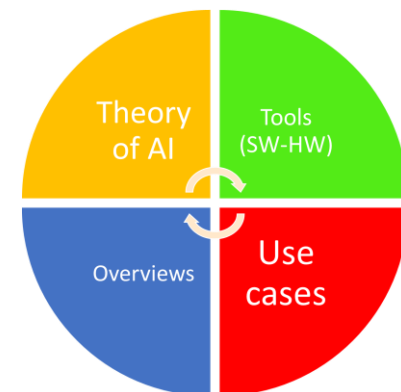


THEORY

the Bias/Variance Tradeoff (Dilemma)

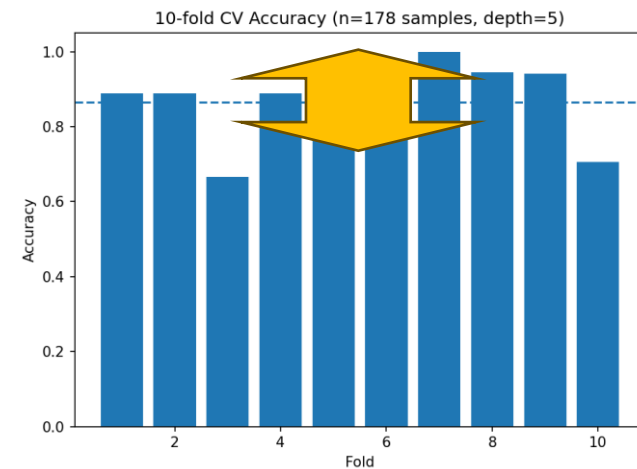


One thing to know for all your life as an AI expert!



Empirical illustration on the Wine dataset

We process not only the mean error but also the variance of the errors in the 10 tests



```
# 5. (optional) print metrics
print("Training accuracy:", round(train_accuracy, 3))
print("Cross-validation accuracies:", np.round(cv_scores, 3))
print("Mean CV accuracy:", round(cv_mean, 3))
print("Variance of CV accuracies:", round(np.var(cv_scores), 3))
```

```
Training accuracy: 1.0
Cross-validation accuracies: [0.889 0.889 0.611 0.889 0.833 0.833 1.      0.944 0.941 0.765]
Mean CV accuracy: 0.859
Variance of CV accuracies: 0.011
```

How is changing the mean and std of the CV error with deeper trees?

- **Experiment**

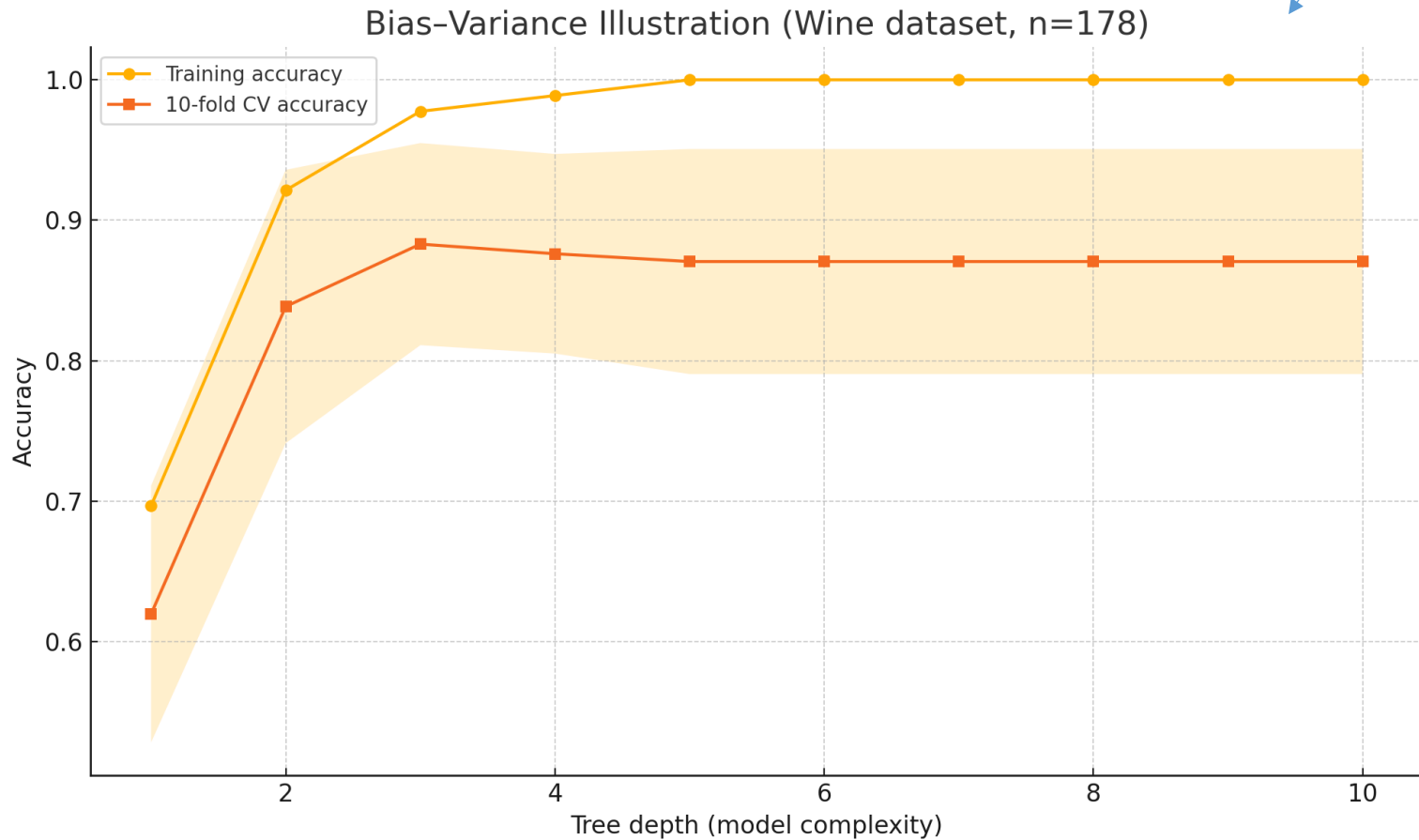
Decision-Tree depths 1 $\rightarrow$ 10, 10-fold CV.

Depth	Train Acc	CV Acc (mean $\pm$ sd)
1	0.70	0.62 $\pm$ 0.09
3	0.98	0.88 $\pm$ 0.07 $\leftarrow$ optimal
5	1.00	0.87 $\pm$ 0.08
≥ 6	1.00	0.87 $\pm$ 0.08

- Training accuracy climbs monotonically.
- Validation accuracy peaks then flattens/declines $\rightarrow$ classic U-shape (see figure).
- At depth 5+ the tree **memorises** (bias $\approx$ 0) but gains **no extra generalisation**.

Bias–Variance Illustration (Wine dataset, n=178)

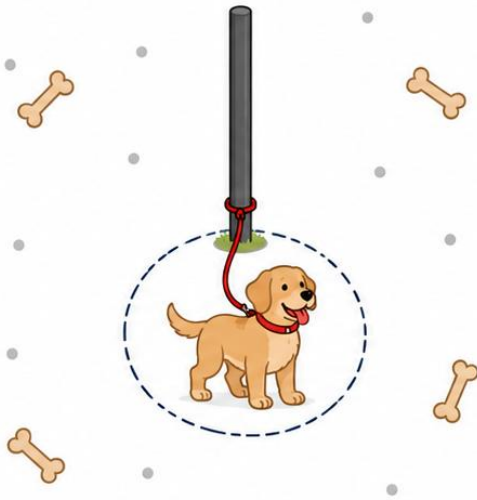
Depth	Train Acc	CV Acc (mean $\pm$ sd)
1	0.70	0.62 ± 0.09
3	0.98	0.88 ± 0.07 ← optimal
5	1.00	0.87 ± 0.08
≥ 6	1.00	0.87 ± 0.08



The bias-variance dilemma

High Bias

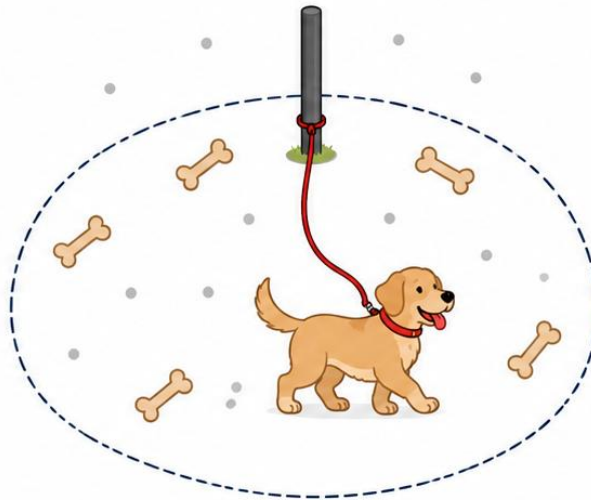
Underfitting



Model too simple.
Misses the true pattern.

Good Tradeoff

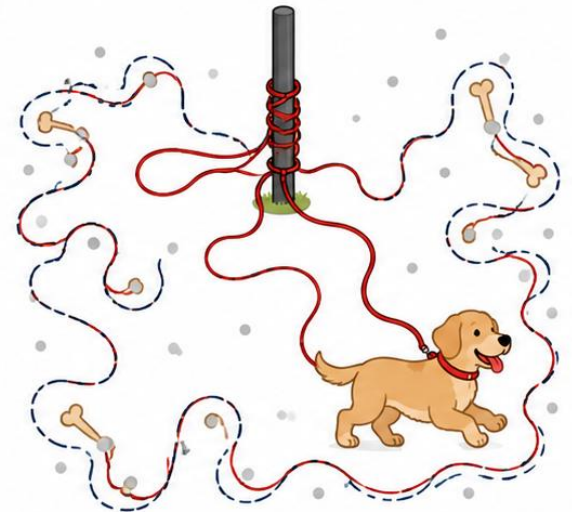
Good Generalization



Balanced complexity.
Captures the overall pattern.

High Variance

Overfitting



Model too complex.
Fits noise and details.



Too simple misses structure. Too complex fits noise. **The goal is the best generalization on unseen data.**

The bias-variance dilemma (regressor)

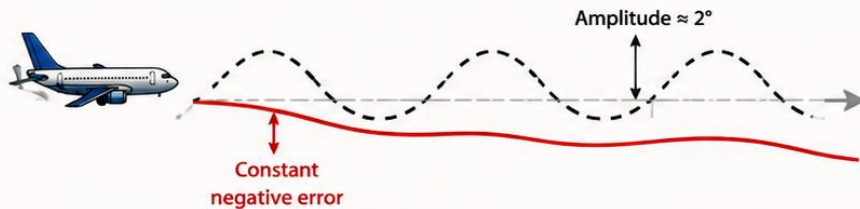
THE BIAS-VARIANCE DILEMMA IN AN AUTOPILOT SYSTEM

--- Expected route (ideal)

— Real route (measured)

HIGH BIAS – LOW VARIANCE

Automatic System 1



BIAS

The system is consistently below the expected route (underfitting)

- Real route is too smooth
- It does not capture the required amplitude
- Error is large and systematic
- Low sensitivity to disturbances (very stable)

AUTOPILOT SYSTEM



SYSTEM ERROR

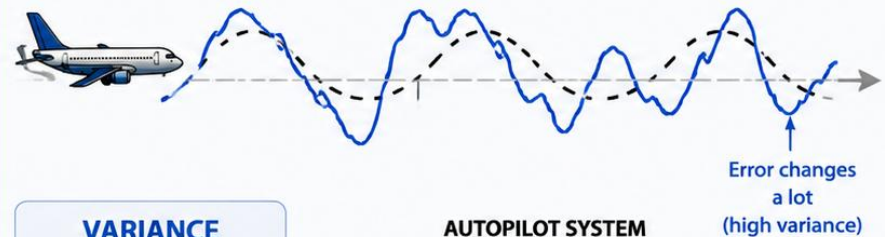
Error = Real route – Expected route

We want the error to be:

- Small on average (low bias)
- Stable across time (low variance)

LOW BIAS – HIGH VARIANCE

Automatic System 2



VARIANCE

The system oscillates above and below the expected route (overfitting)

- Real route follows the average trend
- It captures the amplitude
- Error changes a lot and is not stable
- High sensitivity to disturbances

AUTOPILOT SYSTEM



SYSTEM ERROR

Error = Real route – Expected route

We want the error to be:

- Small on average (low bias)
- Stable across time (low variance)



GOAL: Choose the autopilot system that achieves the best tradeoff: **low bias and low variance** for **safe** and accurate flight.

Bias–Variance Trade-off: the theory

Prediction error decomposition

- $\mathbb{E}[\cdot]$ “expected value” – the average you would obtain if you could repeat the experiment many times.

$$\mathbb{E}[(y - \hat{y})^2] = \underbrace{(\text{Bias}[\hat{y}])^2}_{\text{systematic error}} + \underbrace{\text{Var}[\hat{y}]}_{\text{sensitivity to data}} + \underbrace{\sigma_{\text{noise}}^2}_{\text{irreducible}}$$

High bias → under-fitting:

the model is too simple, so its average prediction stays far from the true function.

High variance → over-fitting:

the model is too complex, so its predictions swing wildly between datasets; the sweet spot balances the two to minimize expected error.

What are the mean() and std() of the kFCV

Let $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}[\varepsilon] = \sigma_{\text{noise}}^2$.

This part is not in the exam

$$\begin{aligned}
 \mathbb{E}_{\mathcal{D}, \varepsilon}[(y - \hat{y})^2] &= \mathbb{E}_{\varepsilon}[\varepsilon^2] + \mathbb{E}_{\mathcal{D}}[(f(x) - \hat{y})^2] \\
 &= \sigma_{\text{noise}}^2 + \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{y}] - f(x)\right)^2}_{\text{Bias}[\hat{y}]^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{y} - \mathbb{E}_{\mathcal{D}}[\hat{y}])^2]}_{\text{Var}[\hat{y}]} \\
 &= \sigma_{\text{noise}}^2 + \underbrace{\text{Bias}[\hat{y}]^2}_{\text{can } \downarrow \text{ with better model}} + \underbrace{\text{Var}[\hat{y}]}_{\text{can } \downarrow \text{ with simpler model}}
 \end{aligned}$$

$\mathcal{D}$ = different datasets partition

$\mathbb{E}_{\mathcal{D}}[\bullet]$ = average prediction

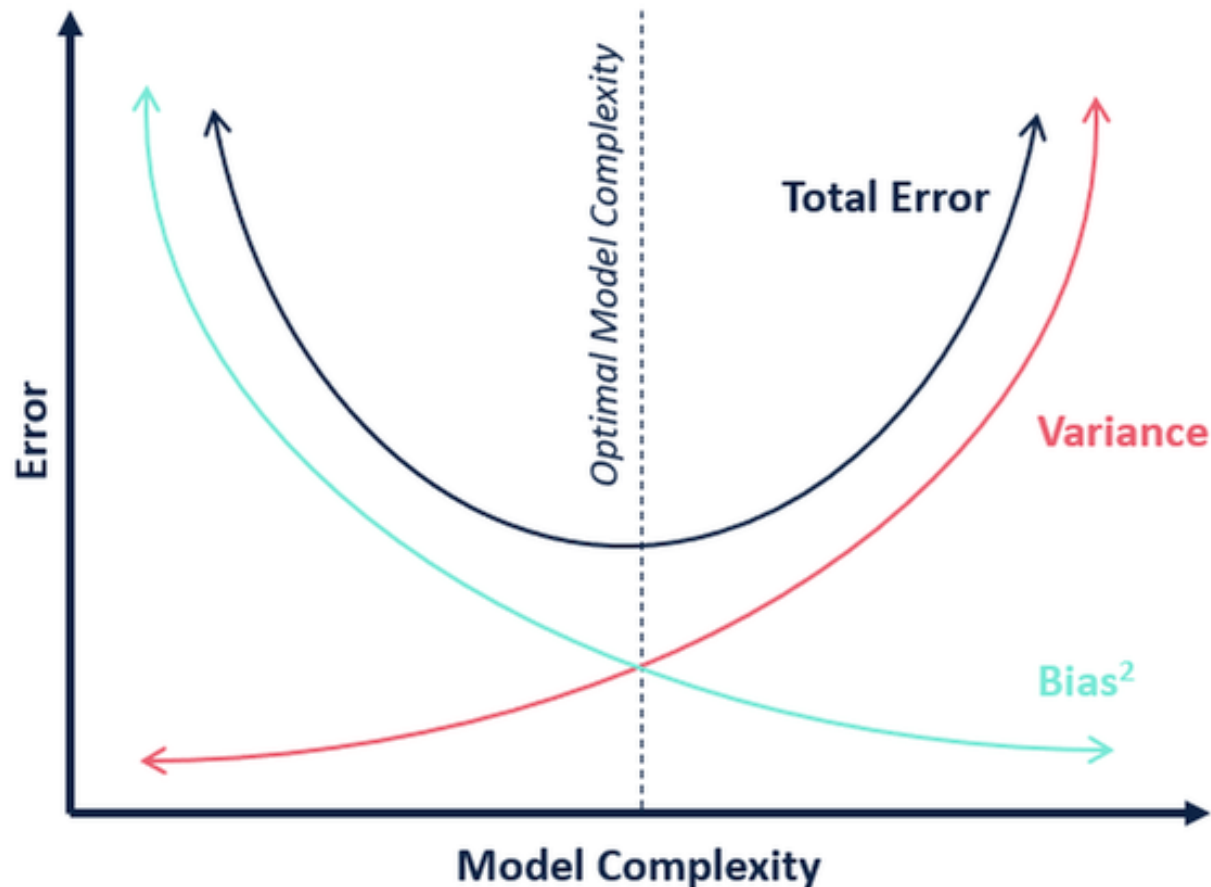
Depth	Train Acc	CV Acc (mean $\pm$ sd)
1	0.70	0.62 $\pm$ 0.09
3	0.98	0.88 $\pm$ 0.07 $\leftarrow$ optimal
5	1.00	0.87 $\pm$ 0.08
≥ 6	1.00	0.87 $\pm$ 0.08

It more intuitive to monitor the average error and the std in the kFCV.

The kFCV is a Monte Carlo simulation of the prediction error

$$\mathbb{E}[(y - \hat{y})^2] = \text{Bias}^2(x) + \text{Var}(x) + \sigma_{\text{noise}}^2$$

Visualization of the Bias/Tradeoff Tradeoff

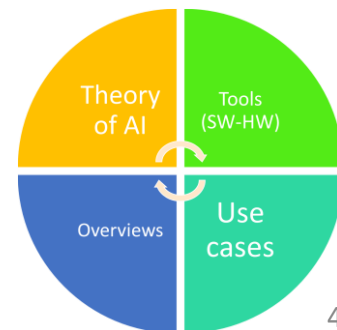
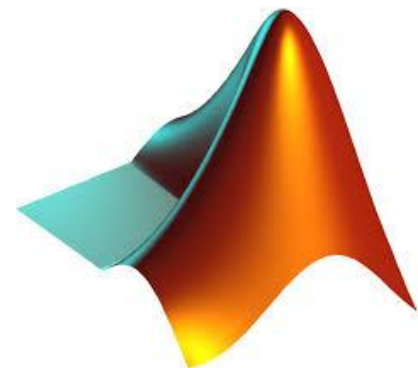


Not time nor epochs, it's complexity!



Toolboxes

Loading,
EDA/plotting
in **Matlab**



Load the dataset (1/2)

% Load the dataset and cast data

```
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data';
```

```
filename = 'iris.dat';
```

```
outfilename = websave(filename,url);
```



5.1000	3.5000	1.4000	0.2000
4.9000	3	1.4000	0.2000
4.7000	3.2000	1.3000	0.2000
4.6000	3.1000	1.5000	0.2000
5	3.6000	1.4000	0.2000
5.4000	3.9000	1.7000	0.4000
4.6000	3.4000	1.4000	0.3000
5	3.4000	1.5000	0.2000
4.4000	2.9000	1.4000	0.2000
4.9000	3.1000	1.5000	0.1000
5.4000	3.7000	1.5000	0.2000
4.8000	3.4000	1.6000	0.2000
4.8000	3	1.4000	0.1000
4.3000	3	1.1000	0.1000

...

```
fid = fopen(outfilename, 'r');
```

```
S = textscan(fid,'%f%f%f%f%s' , 'delimiter' , ',' );
```

```
fclose(fid);
```

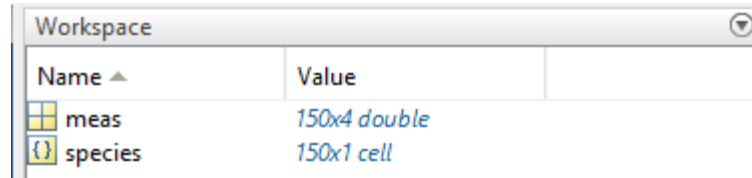
```
meas = [S{1}, S{2}, S{3}, S{4}];
```

```
species = S{5};
```

% Equivalent to ">>load fisheriris"

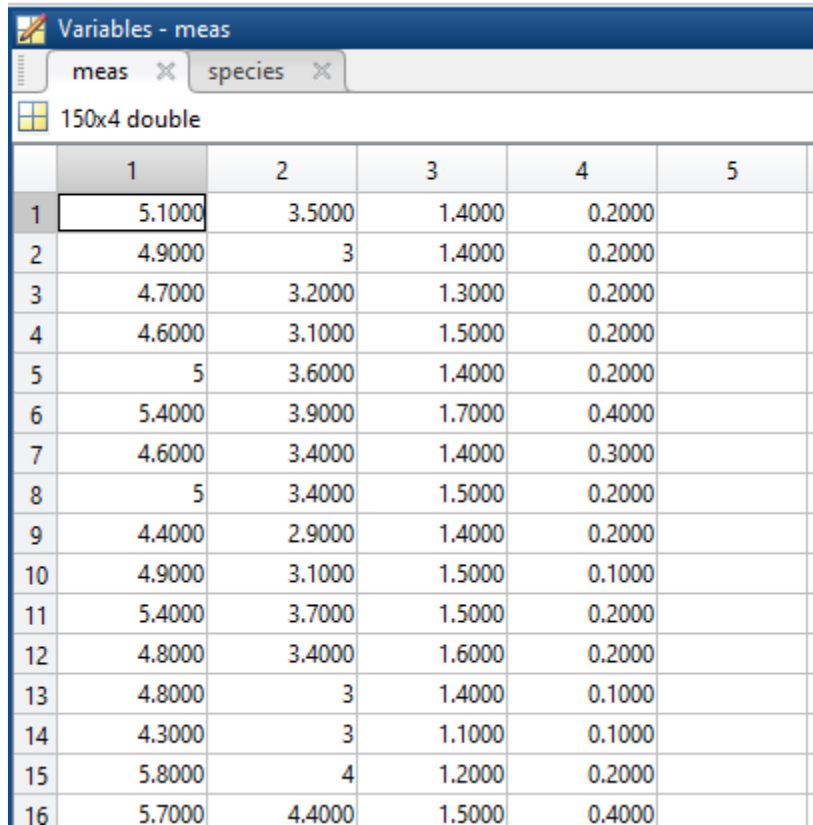
meas = «measurements»

Load the dataset (2/2)



Workspace

Name	Value
meas	150x4 double
species	150x1 cell

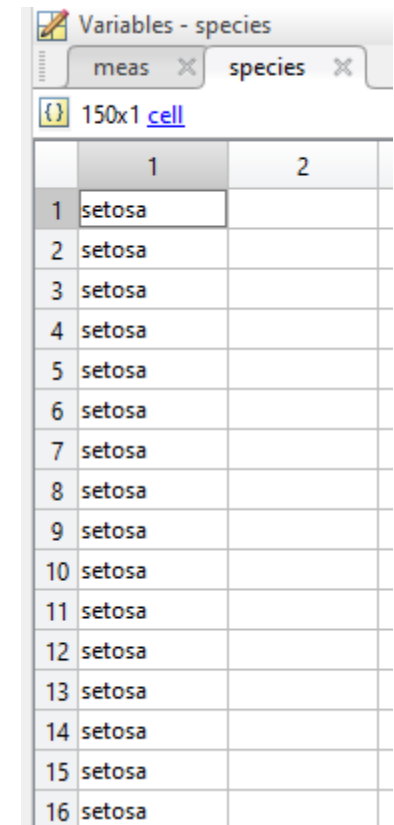


Variables - meas

meas species

150x4 double

	1	2	3	4	5
1	5.1000	3.5000	1.4000	0.2000	
2	4.9000	3	1.4000	0.2000	
3	4.7000	3.2000	1.3000	0.2000	
4	4.6000	3.1000	1.5000	0.2000	
5	5	3.6000	1.4000	0.2000	
6	5.4000	3.9000	1.7000	0.4000	
7	4.6000	3.4000	1.4000	0.3000	
8	5	3.4000	1.5000	0.2000	
9	4.4000	2.9000	1.4000	0.2000	
10	4.9000	3.1000	1.5000	0.1000	
11	5.4000	3.7000	1.5000	0.2000	
12	4.8000	3.4000	1.6000	0.2000	
13	4.8000	3	1.4000	0.1000	
14	4.3000	3	1.1000	0.1000	
15	5.8000	4	1.2000	0.2000	
16	5.7000	4.4000	1.5000	0.4000	



Variables - species

meas species

150x1 cell

	1	2
1	setosa	
2	setosa	
3	setosa	
4	setosa	
5	setosa	
6	setosa	
7	setosa	
8	setosa	
9	setosa	
10	setosa	
11	setosa	
12	setosa	
13	setosa	
14	setosa	
15	setosa	
16	setosa	

% Problem

P = meas;

% Targets

T = zeros(size(species));

for i = 1 : size(T,1)

 switch species{i}

 case 'setosa'

 T(i) = 0;

 case 'versicolor'

 T(i) = 1;

 case 'virginica'

 T(i) = 2;

 end

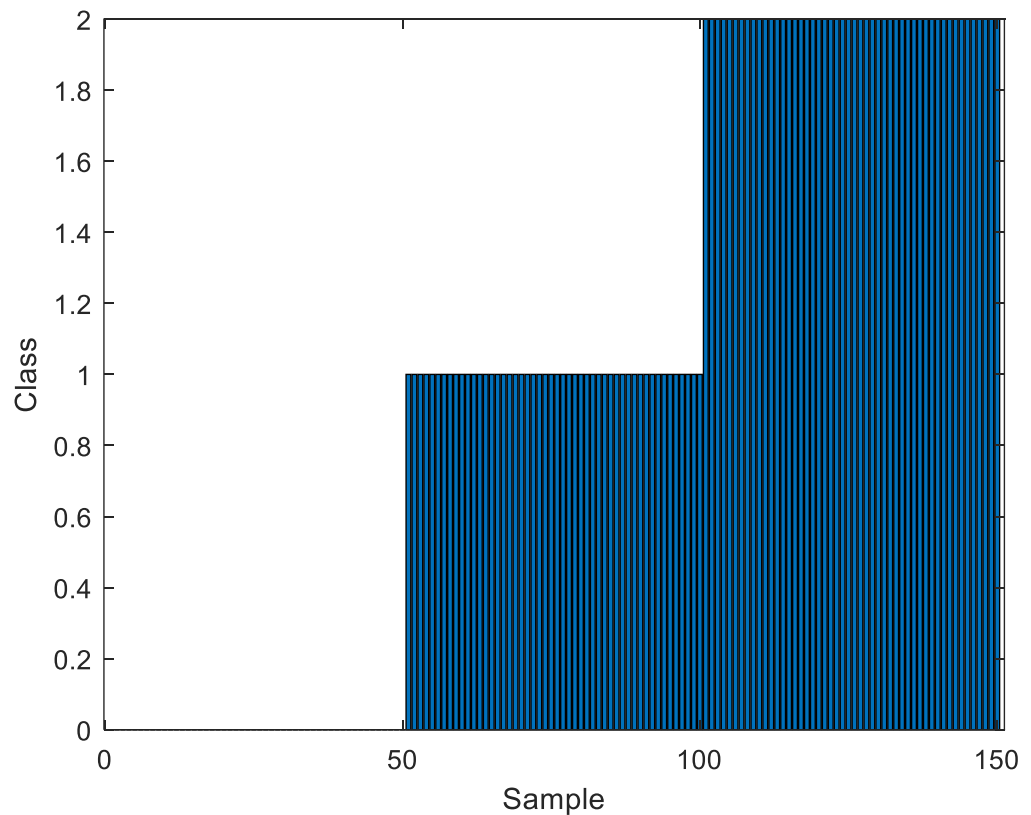
end

figure; bar(T);

 xlabel('Sample');

 ylabel('Class')

Create the data structures



Plotmatrix() $\rightarrow$ Feat_i VS Feat_j

%% creating the feature matrix Problem (X in the course)

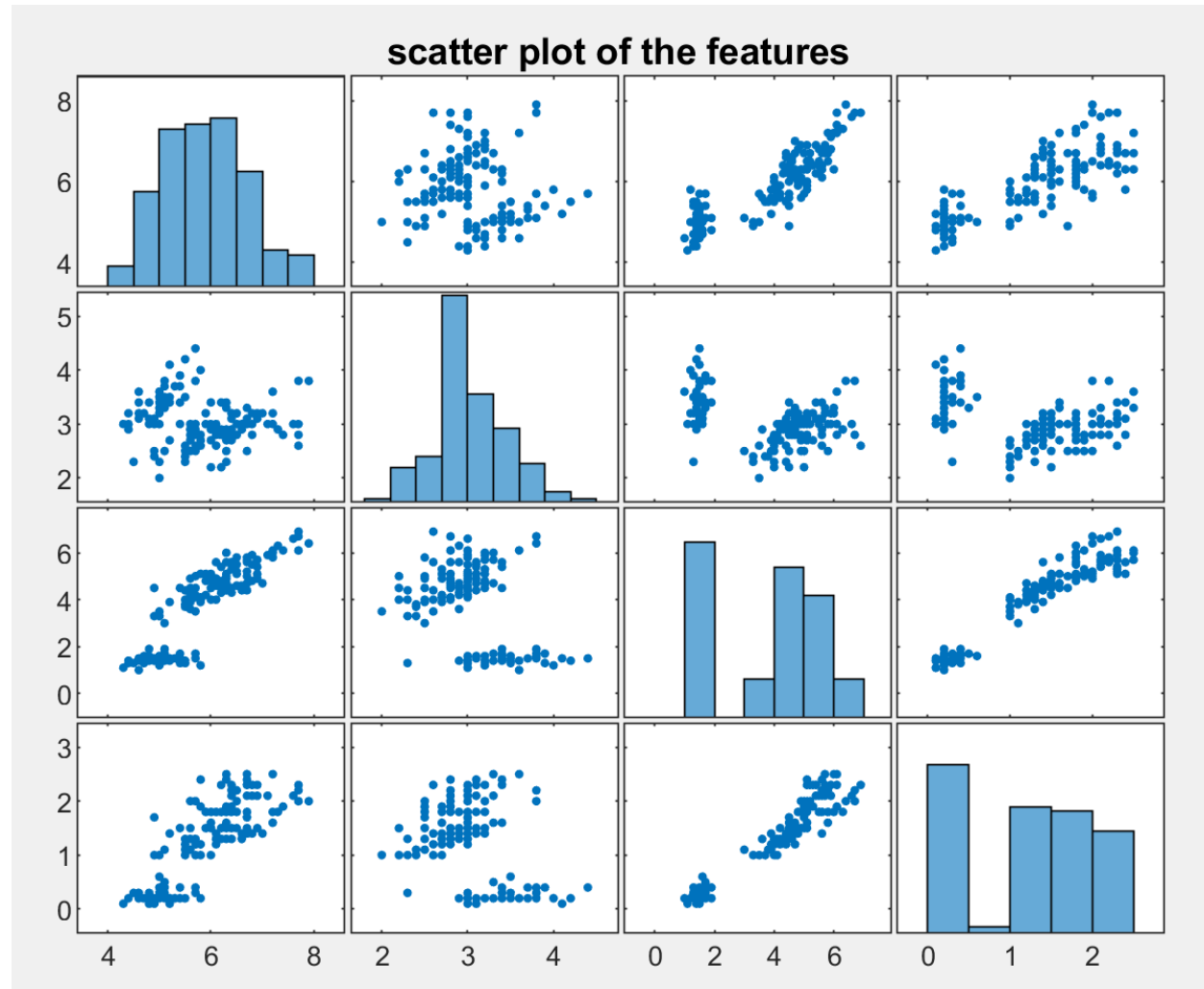
P = meas;

% plot the feature matrix

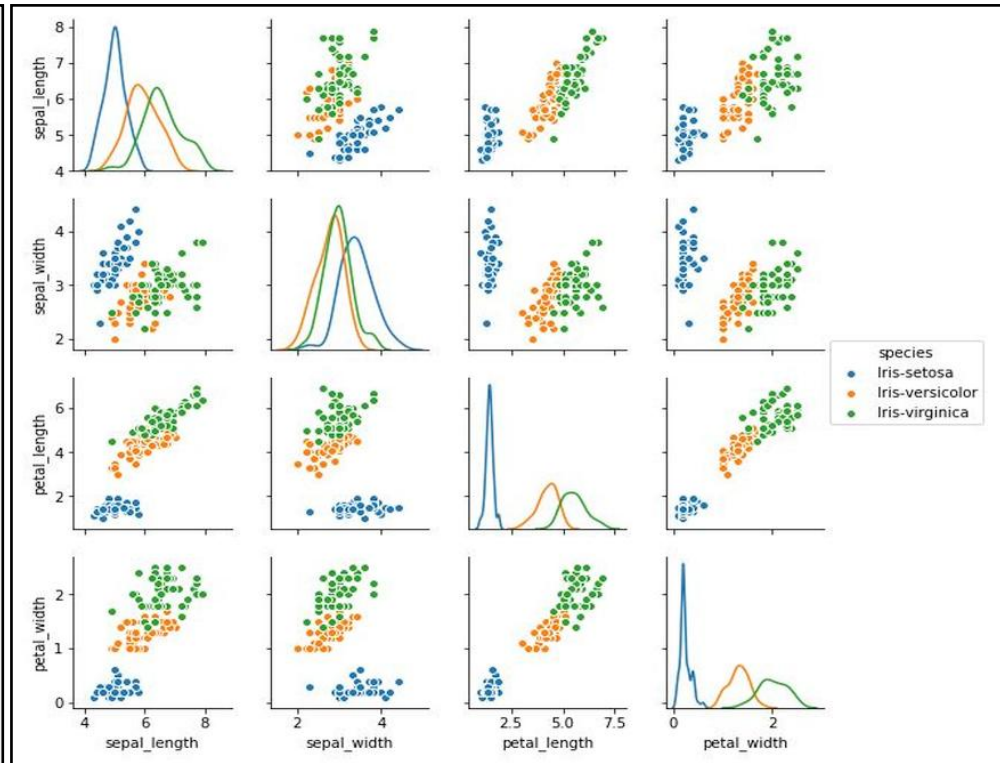
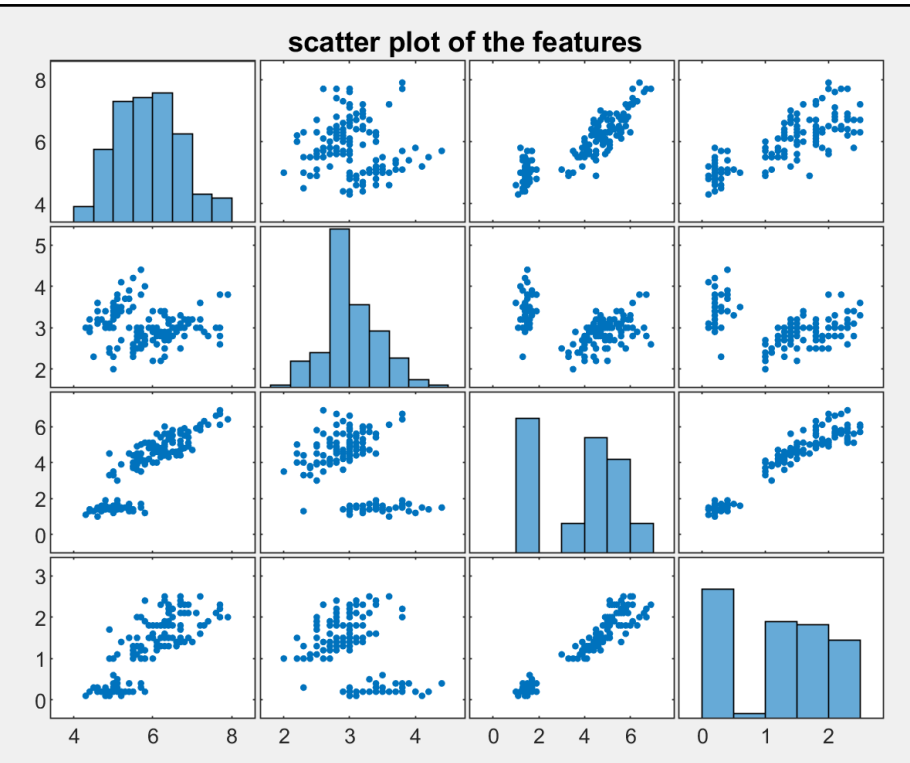
figure

plotmatrix(P)

title('scatter plot of the features')

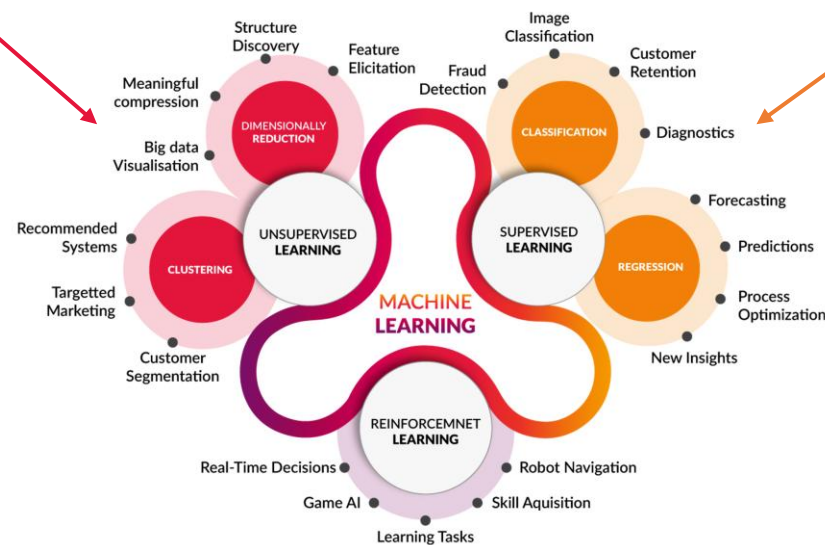
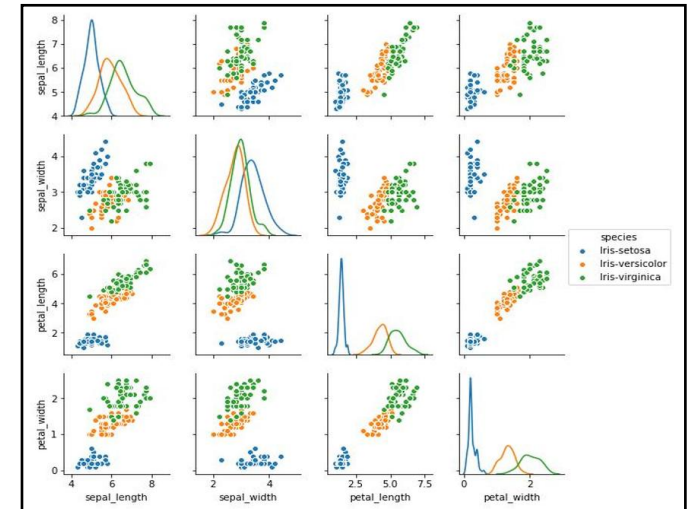
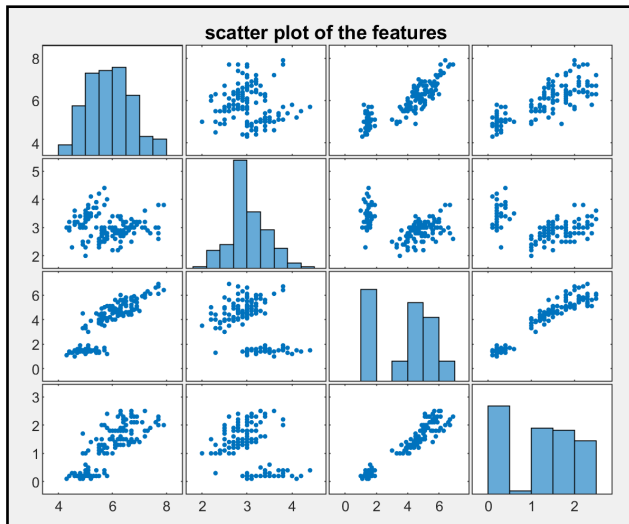


plotmatrix() VS pairplot ()

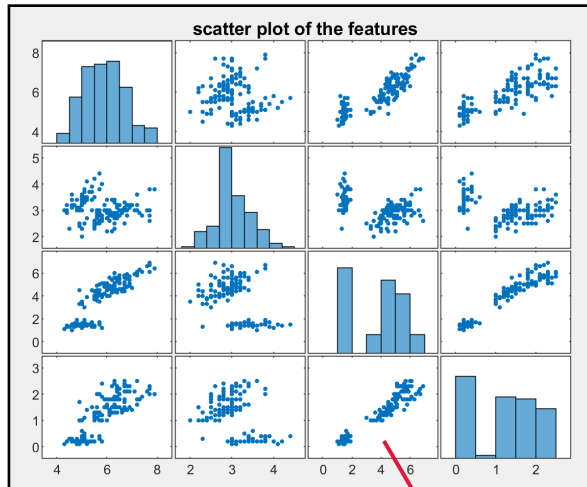


Unsupervised VS Supervised

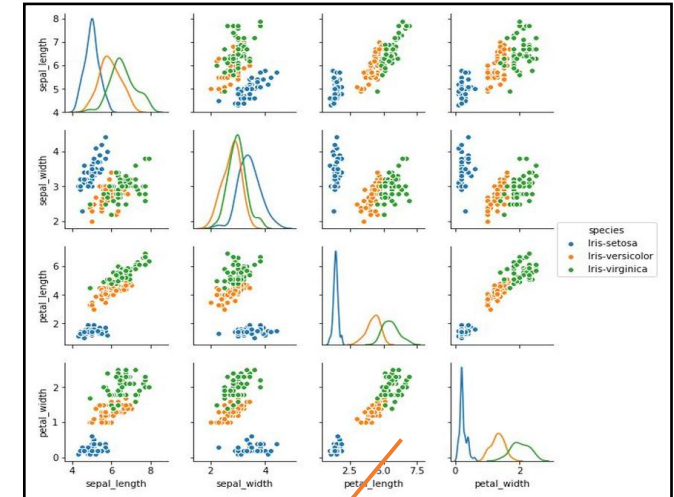
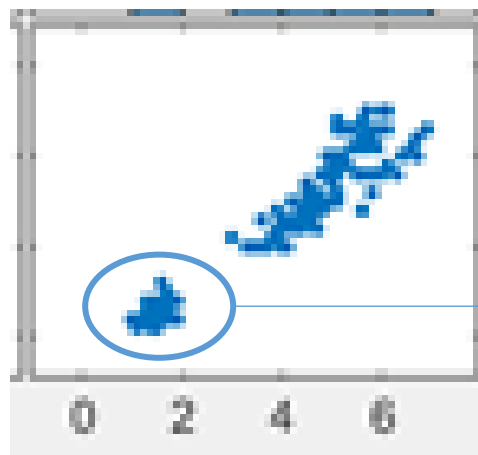
Unsupervised VS Supervised



Clusters in the unsupervised analysis

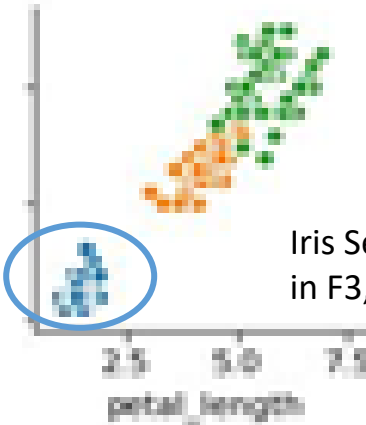


F4 VS F3



Petal with VS Petal length

Iris Setosa Cluster!
in F3,F4 subspace is well separate



Preliminary data analysis

```
statisticsP = zeros(4, size(P,2));
statisticNames{1} = 'Mean';
statisticNames{2} = 'Std ';
statisticNames{3} = 'Min ';
statisticNames{4} = 'Max ';
statisticsP(1,:) = mean(P);
statisticsP(2,:) = std(P);
statisticsP(3,:) = min(P);
statisticsP(4,:) = max(P);
```

```
fprintf('\t\tFeat 1\tFeat 2\tFeat 3\tFeat 4\n');
for i = 1 : 4
    fprintf('%s\t', statisticNames{i});
    for j = 1 : size(P,2)
        fprintf('%03.03ft', statisticsP(i,j));
    end
    fprintf('\n')
end
```

Matlab is natively working on columns

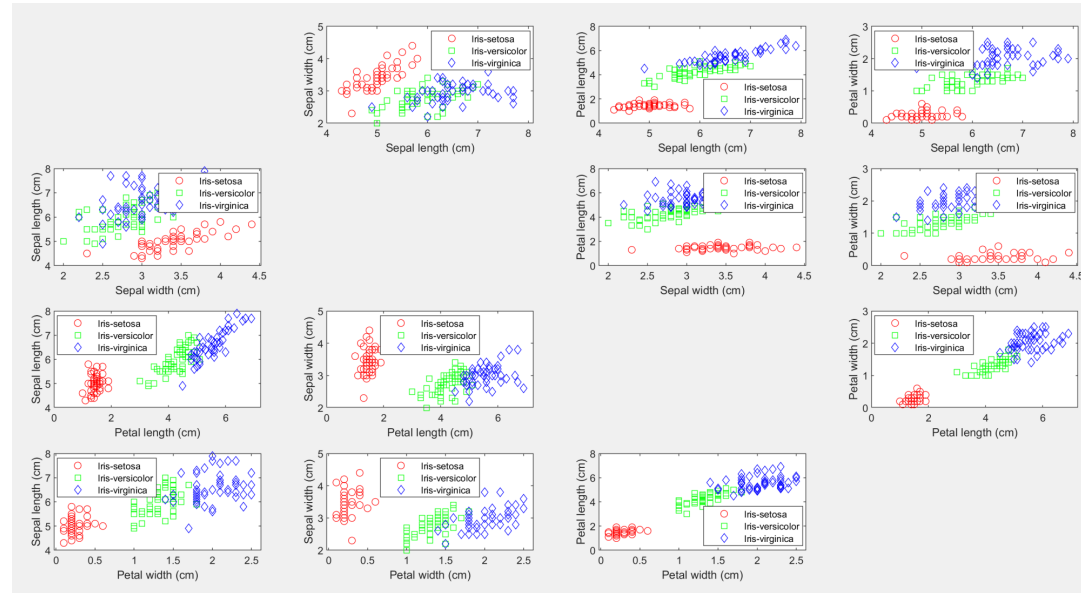
5.1000	3.5000	1.4000	0.2000
4.9000	3	1.4000	0.2000
4.7000	3.2000	1.3000	0.2000
4.6000	3.1000	1.5000	0.2000
5	3.6000	1.4000	0.2000
5.4000	3.9000	1.7000	0.4000
4.6000	3.4000	1.4000	0.3000
5	3.4000	1.5000	0.2000
4.4000	2.9000	1.4000	0.2000
4.9000	3.1000	1.5000	0.1000
5.4000	3.7000	1.5000	0.2000
4.8000	3.4000	1.6000	0.2000
4.8000	3	1.4000	0.1000
4.3000	3	1.1000	0.1000

mean(P) → [5.843 3.057 3.758 1.199]

Command Window				
New to MATLAB? See resources for Getting Started .				
	Feat 1	Feat 2	Feat 3	Feat 4
Mean	5.843	3.057	3.758	1.199
Std	0.828	0.436	1.765	0.762
Min	4.300	2.000	1.000	0.100
Max	7.900	4.400	6.900	2.500
fx >>				

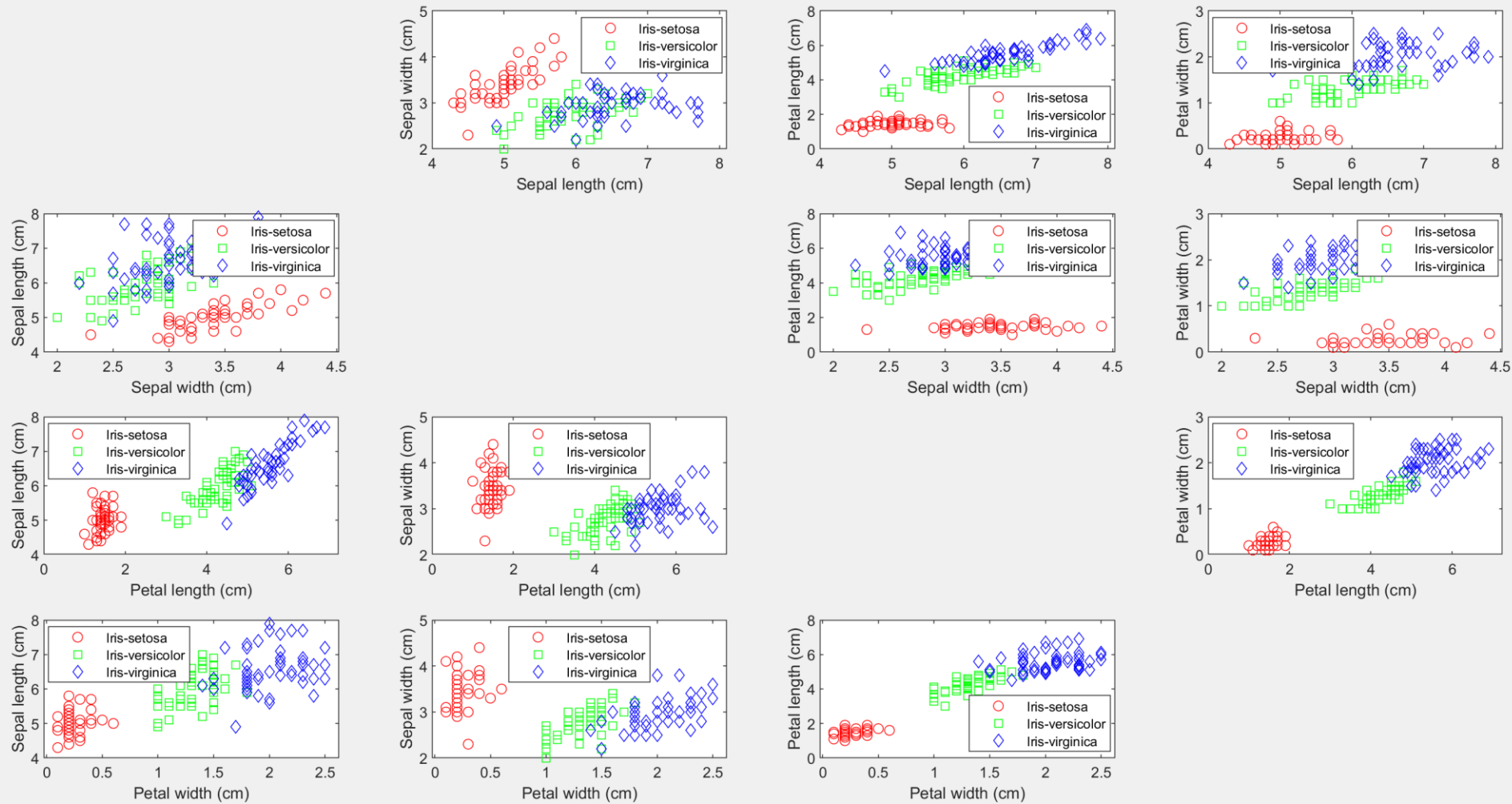
Plot the data (1/2)

```
% Plot the dataset
variableNames{1} = 'Sepal length (cm)';
variableNames{2} = 'Sepal width (cm)';
variableNames{3} = 'Petal length (cm)';
variableNames{4} = 'Petal width (cm)';
iCount = 1;
figure
for i = 1 : 4
    for j = 1 : 4
        if (i~=j)
            subplot(4, 4, iCount)
            gscatter(P(:,i), P(:,j), species,'rgb','osd');
            xlabel(variableNames{i})
            ylabel(variableNames{j})
        end
        iCount = iCount + 1;
    end
end
end
```



You can create your own visualization plots and dashboard! This is a valuable part of the job...

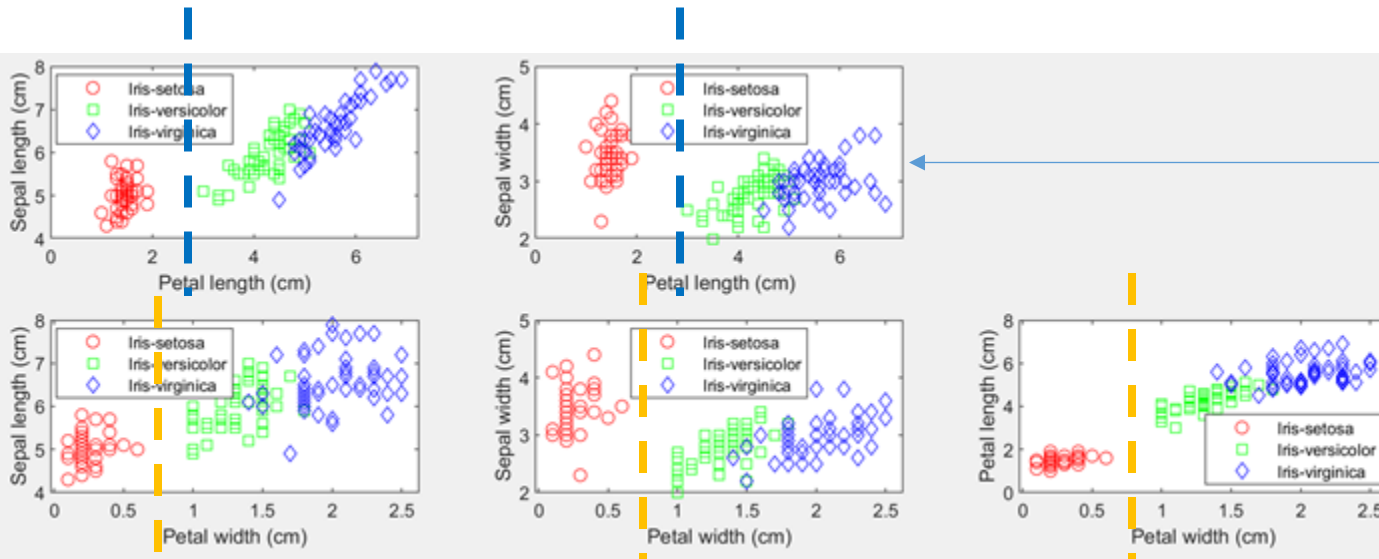
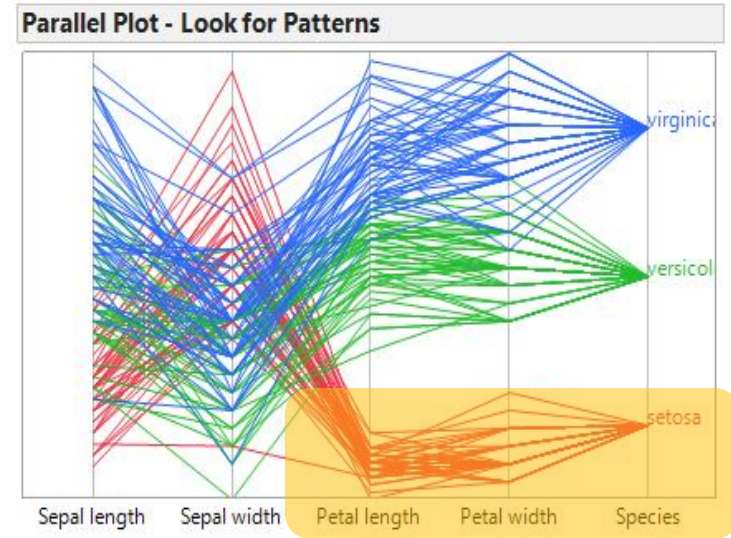
Plot the data (2/2)



EDA: application

From the Parallel Plot we already verified the separation at least of the Setosa Class
Just using the Petal length or Petal width

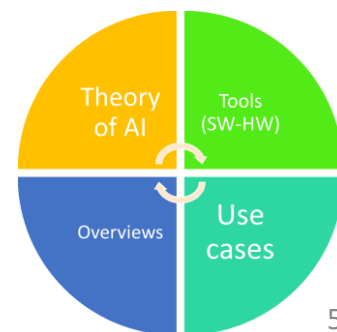
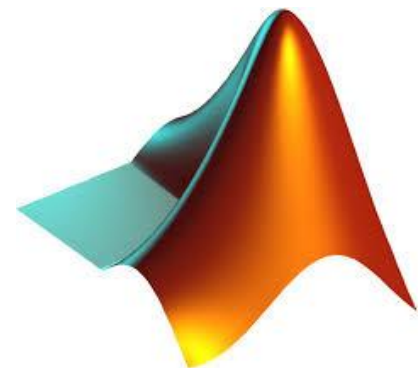
This approach give EXPLANABILITY to your models!





Toolboxes

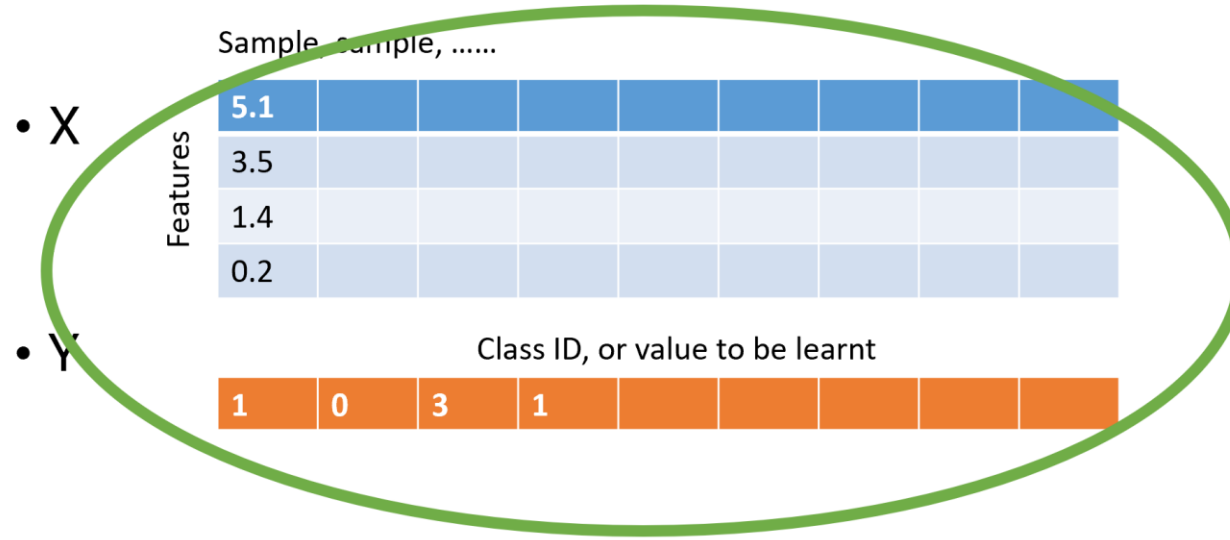
Feature Engineering + Classification in **Matlab**



Recall Feat. Eng. With labels

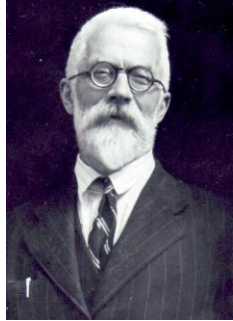
$Y = \text{FUNC}(X)$
FEATURE ENGINEERING (C)

- Optimizing all the dataset at time!



See the next
example: LDA

LDA Classification



1936

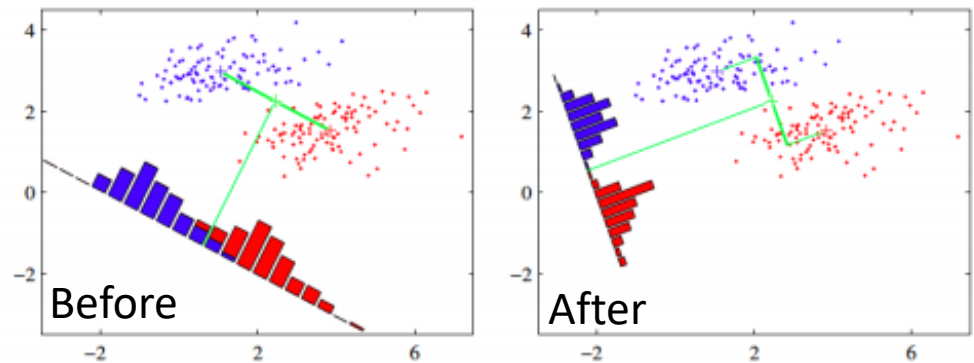
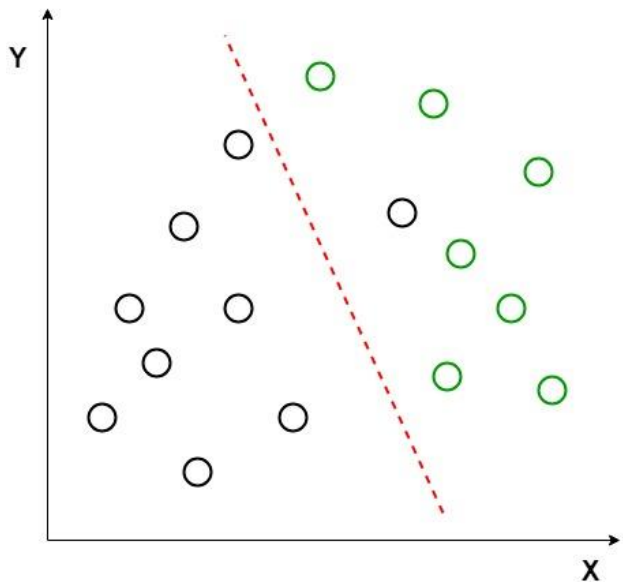
% Linear Discriminant
% Analysis

```
lda = fitcdiscr(P,T);
```

```
ldaClass = resubPredict(lda);
```

Just an example:
we will present this topic
later in the course

Fischer's
Linear Discriminant Analysis Idea in 2D



This method is engineering the features using
not just the distributions, but also the labels

Linear Classification

% Linear Discriminant Analysis

`lda = fitcdiscr(P,T);`

% predict values

% using the predictor data stored in `lda.X`.

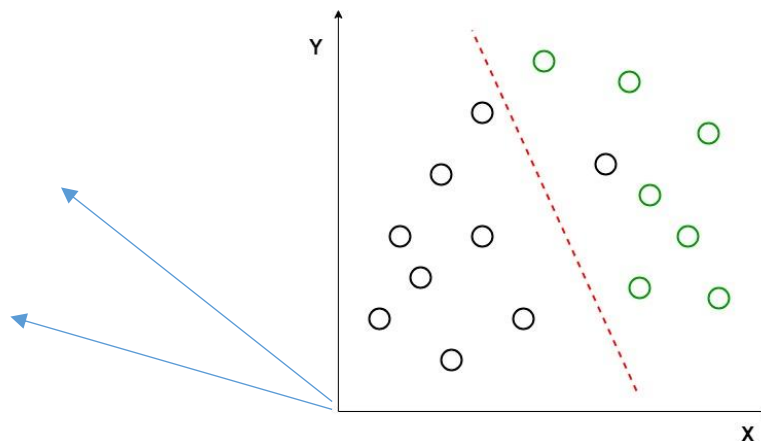
`ldaClass = resubPredict(lda);`

	1	2	3	4
1	5.1000	3.5000	1.4000	0.2000
2	4.9000	3	1.4000	0.2000
3	4.7000	3.2000	1.3000	0.2000
4	4.6000	3.1000	1.5000	0.2000
5	5	3.6000	1.4000	0.2000
6	5.4000	3.9000	1.7000	0.4000
7	4.6000	3.4000	1.4000	0.3000
8	5	3.4000	1.5000	0.2000
9	4.4000	2.9000	1.4000	0.2000
10	4.9000	3.1000	1.5000	0.1000
11	5.4000	3.7000	1.5000	0.2000
12	4.8000	3.4000	1.6000	0.2000
13	4.8000	3	1.4000	0.1000
14	4.3000	3	1.1000	0.1000
15	5.8000	4	1.2000	0.2000
16	5.7000	4.4000	1.5000	0.4000

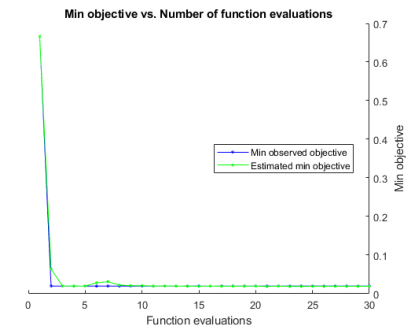
[0
0
0
...
1
1
...]

P

T



Fischer's Linear Discriminant Analysis in 4 dimension



Classification error

% Classification error

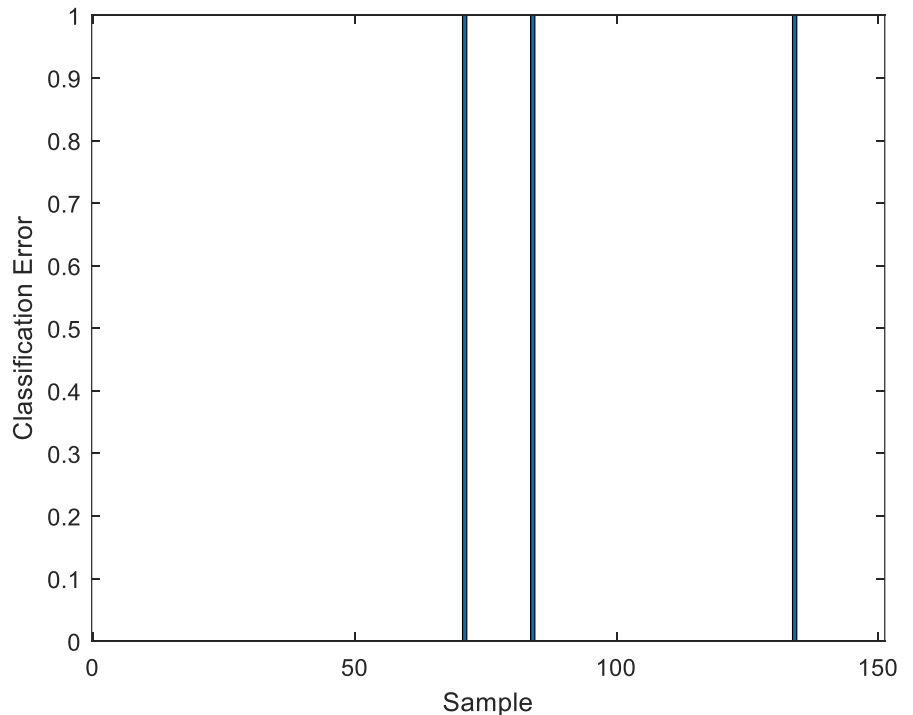
```
errorVector = IdaClass ~= T;
```

```
figure
```

```
bar(errorVector);
```

```
xlabel('Sample')
```

```
ylabel('Classification Error')
```



Figures of merit (1/2)

```
Command Window
New to MATLAB? See resources for Getting Started.

Total classification error = 3
Std of the classification error = 0.140
Total classification error = 2.000 %
Std of the classification error = 14.047 %

confusionMatrix =

    50     0     0
     0    48     1
     0     2    49

fx >>
```

Just an example:
we will present this topic later in the course

Figures of merit

% Figures of merit

```
totalError = sum(errorVector);  
stdError = std(errorVector);  
totalErrorPerc = mean(errorVector)*100;  
stdErrorPerc = stdError * 100;
```

```
fprintf('Total classification error = %d \n', totalError);  
fprintf('Std of the classification error = %03.03f \n', stdError);  
fprintf('Total classification error = %03.03f %% \n', totalErrorPerc);  
fprintf('Std of the classification error = %03.03f %% \n', stdErrorPerc);
```

%Confusion Matrix

```
confusionMatrix = confusionmat(ldaClass,T)
```

Just an example:

we will present this topic later in the course

```
Total classification error = 3  
Std of the classification error = 0.140  
Total classification error = 2.000 %  
Std of the classification error = 14.047 %
```



confusionMatrix =

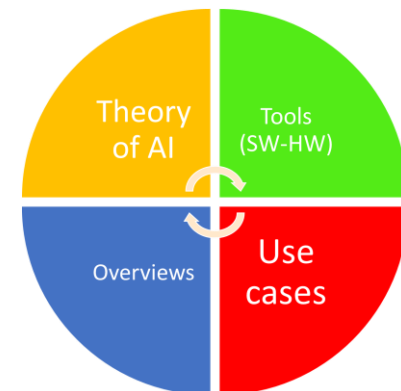
50	0	0
0	48	1
0	2	49



THEORY

New model?

Things to know for a correct start



What to know in case of new models?

- Everyday, new models/learning techniques will be available. How to face this challenge and keep the pace?
- **Consult documentation and examples** before to waste time
 - **Understand the topic, the type, the context!**
 - Check on what libraries models is used
 - General or specific?
 - What the computational complexity of the model?
 - In train
 - In test/deployment
 - Need CUDA?



«Let's use it, we don't need to read documentation!»

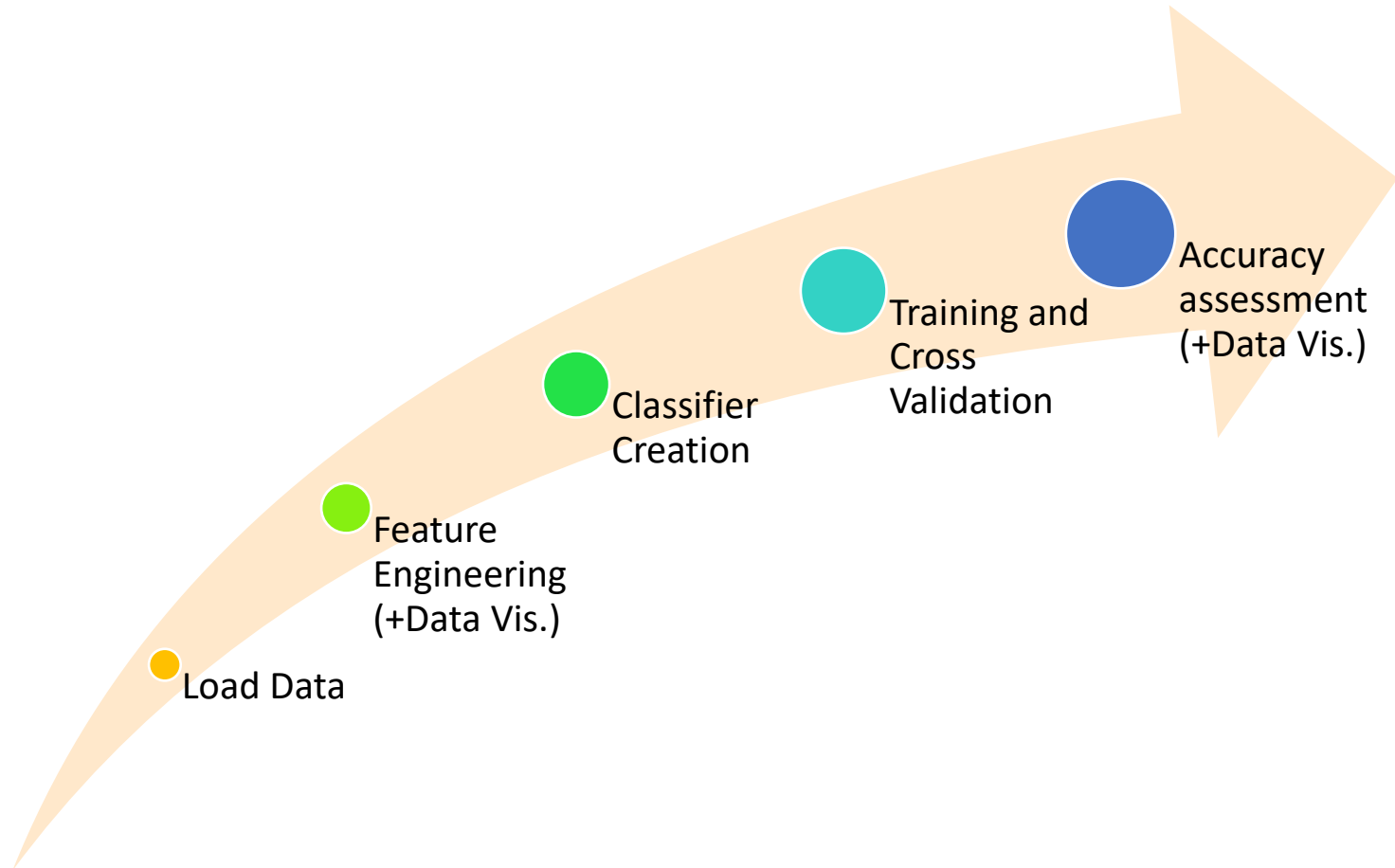
What to know in case of new models?

- Consult documentation and examples before to waste time (cont...)
 - **Memory** needs
 - Training
 - Test
 - Examples/**Dataset size**
1000 sample or 10M samples?
 - **Parameters**
 - Complex parameters to understand?
 - Sensitive to parameter tuning?
 - Can you do just a ***fine-tuning*** or need to re-train?
 - **Portability** on different libraries/environments
 - Is it open/GNU/free or just one single company are running it?

What to do in case of new models?

- Try demos/examples in the documentation before to waste time, checking if it is/not suitable for your applications
- Try new models on a set of standard datasets and compare the
 - a) accuracy and b) computational times
 - Not just the IRIS dataset..
- Remember to Occam's razor!
 - Same performance, but more complex, does it worth it?

Application of a new classifier

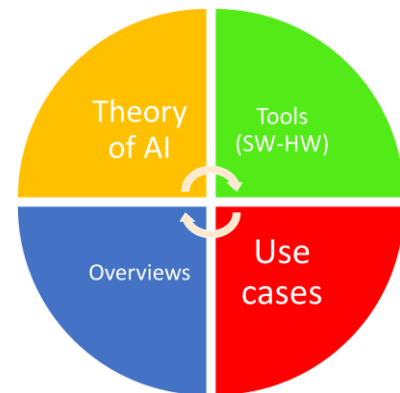




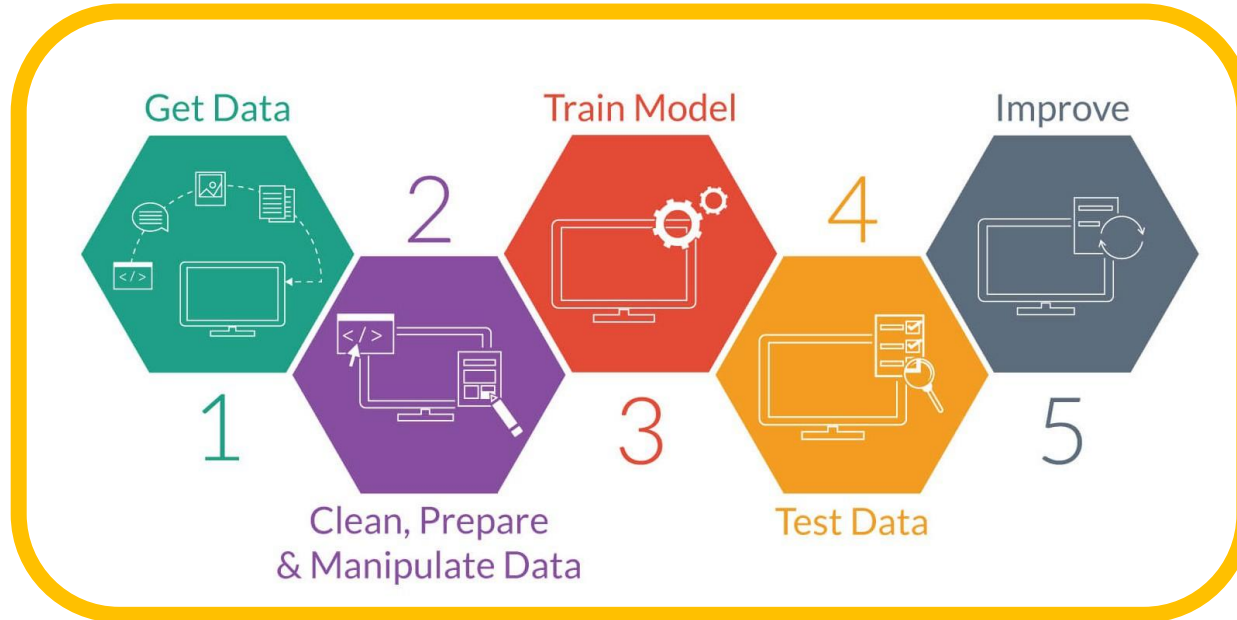
THEORY

A simple example of the complete workflow

The linear regressor

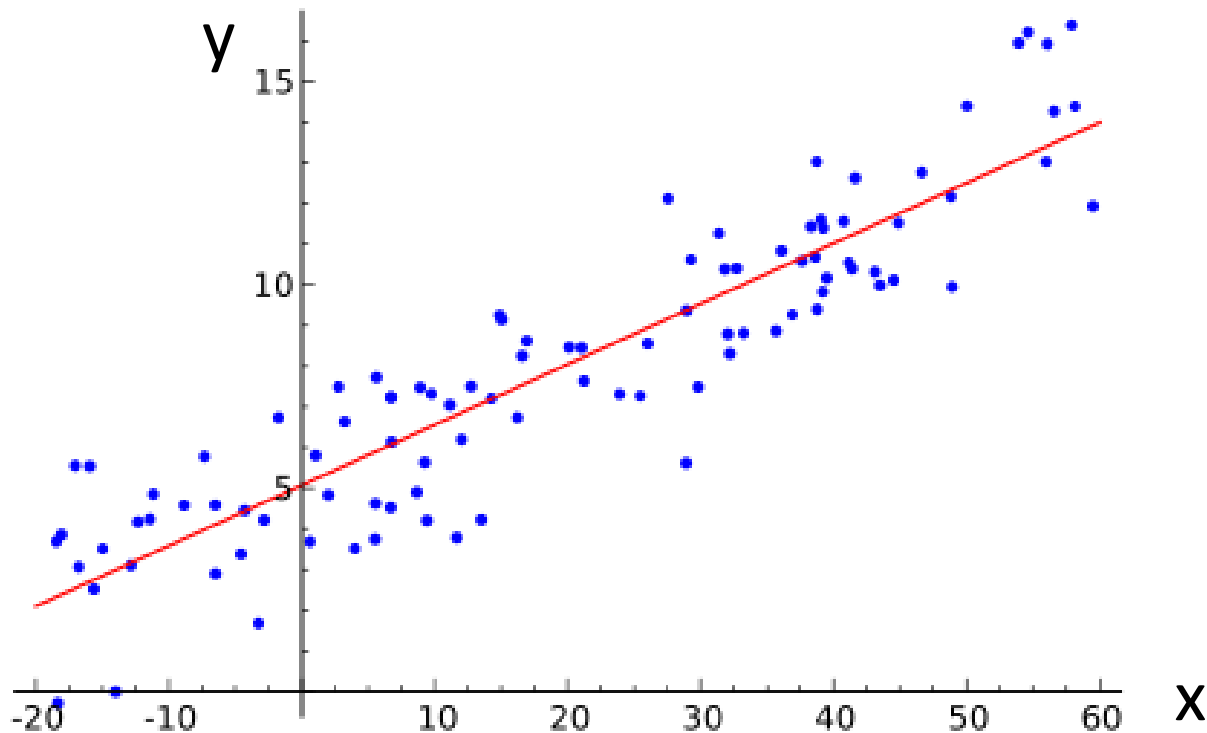


Let's review all the steps of the desing workflow with a simple example



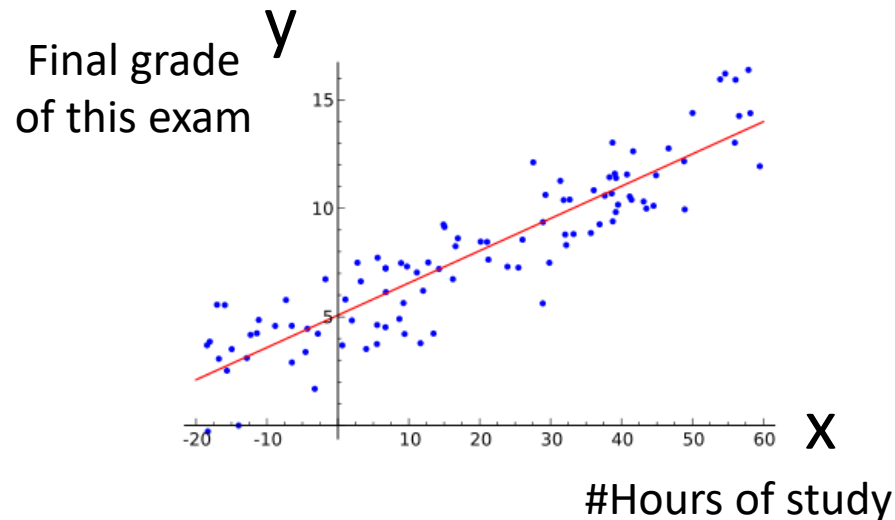
Linear Regression

We want to find the best linear function $y=f(x)$ to explain the data we have



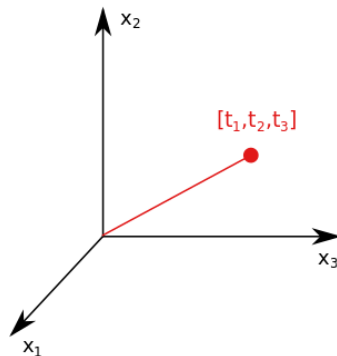
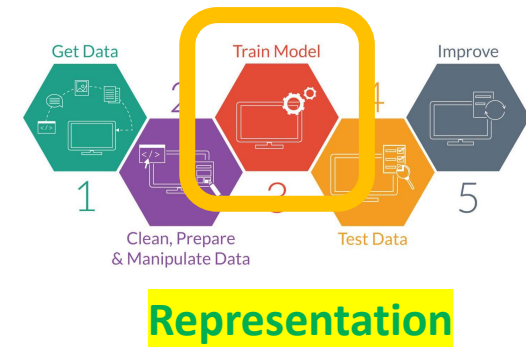
Linear Regression examples

- Age, gender, and diet $\rightarrow$ height of the kid
- Type and grades of college $\rightarrow$ yearly income
- Final grade...



An interesting example...

Extendible to N features



Vectorial form

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$z = [w_1 \ w_2 \ w_3] \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + b$$

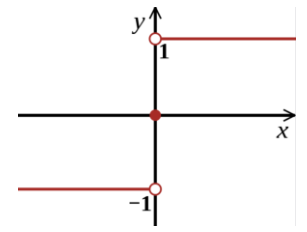
To completely describe the model
you need to fix 4 parameters:

w_1, w_2, w_3, b .

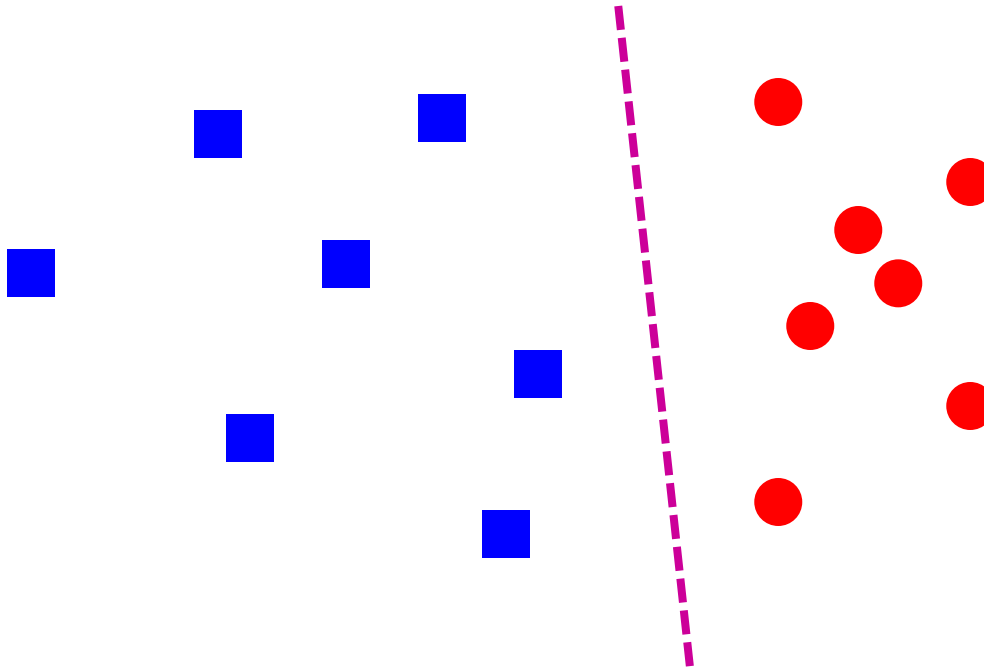
→ You need 4 data points

But you will use much more data!

From regressor to classifiers: Linear Class.



$$\text{sgn}(x) := \begin{cases} -1 & \text{if } x < 0, \\ 0 & \text{if } x = 0, \\ 1 & \text{if } x > 0. \end{cases}$$



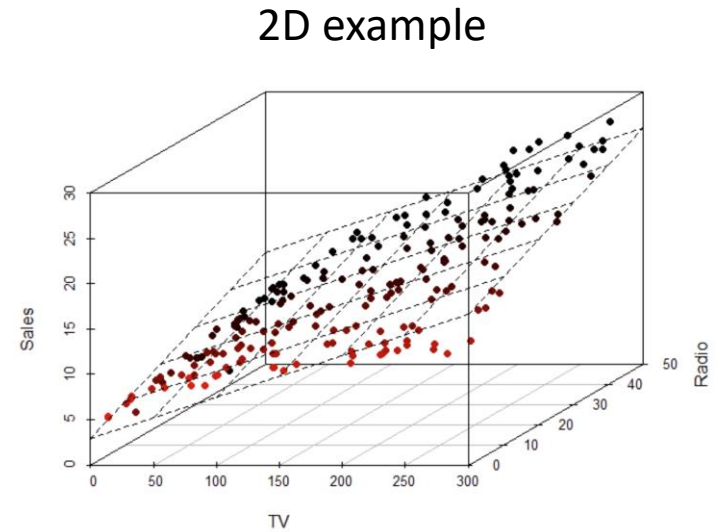
Problem

Find a *linear function* to separate the classes:

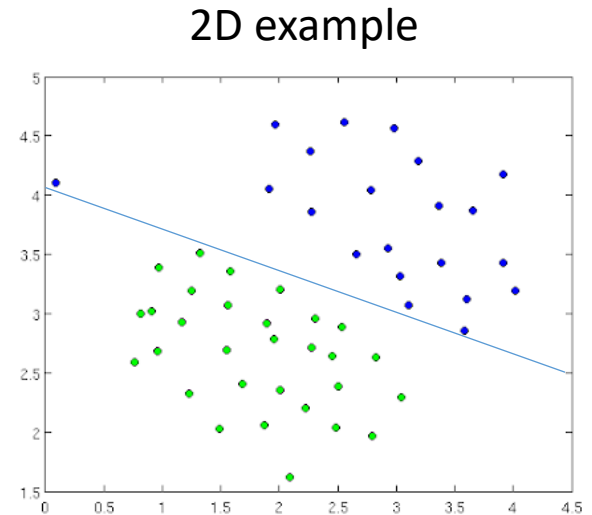
$$f(\mathbf{x}) = \text{sgn}(\underbrace{\mathbf{w} \cdot \mathbf{x} + b}_{\text{Linear regressor}})$$

Linear regressor

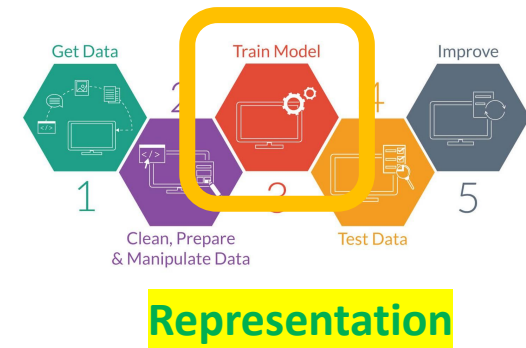
Linear regressor
 $f_1(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$



Linear classifier:
 $f_2(\mathbf{x}) = \text{sgn}(\underbrace{\mathbf{w} \cdot \mathbf{x} + b}_{\text{Linear regressor}})$



Linear Regression



The predicted value of y is given by:

$$\hat{y} = \hat{\beta}_0 + \sum_{j=1}^p X_j \hat{\beta}_j$$

The vector of coefficients $\hat{\beta}$ is the regression model.

JUST A NOTE about computation:

If you add a term in the inputs $X_0 = 1$, the formula becomes a simple matrix product (a compact notation): $\hat{\mathbf{y}} = \mathbf{X} \hat{\boldsymbol{\beta}}$

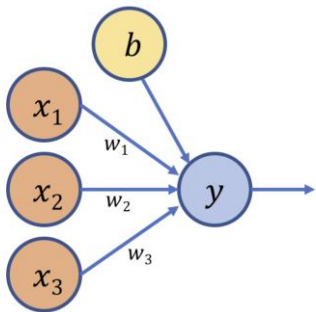
The algebra to get the solution is more easy to deal with.

About notation...

Elements

$$\hat{y} = \hat{\beta}_0 + \sum_{j=1}^p X_j \hat{\beta}_j$$

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$



3 inputs

That is (also)
the linear output
of a neuron

Vectorial form

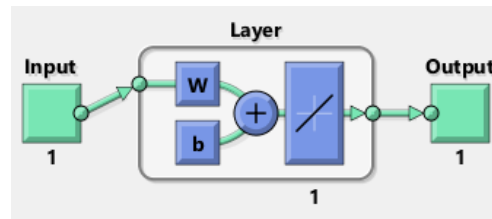
$$\mathbf{z} = \mathbf{w} \cdot \mathbf{x} + \mathbf{b}$$

«**weights**»

Since they multiply
the information coming
from the inputs

«**bias**»

Something
always present,
not modulated
from the input



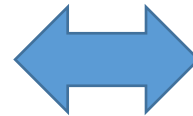
1 inputs

Linear Regression, #Features>1

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

We can write all of the input samples from the training in a single matrix $\mathbf{X}$:

$$\mathbf{X} = \begin{pmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{pmatrix}$$

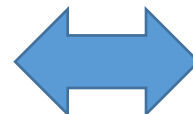


	Individual #1								Individual #n
Age,	5.1								
gender,	3.5								
diet,	1.4								
....	0.2								

n = # samples (**distinct observations**)

m = # features

$$\mathbf{y} = (y_1, y_2, \dots, y_n)$$



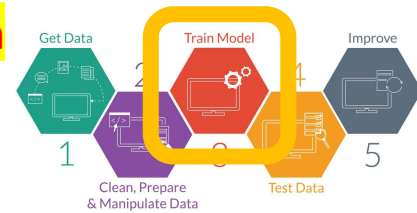
Glucose 3.0

Vector $\mathbf{Y}$ does not contain
classes here!!

We are creating a regressor
→ we will have numbers in $\mathbf{Y}$

Residual Sum-of-Squares

Optimization



To determine the model parameters $\hat{\beta}$ (*weights*) from some data, we minimize the Residual Sum of Squares which is

$$\text{RSS}(\beta) = \sum_{i=1}^N (y_i - \beta x_i)^2$$
$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}$$

Appendix (non in the exam) using the «compact notation» :

or symbolically $\text{RSS}(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)$. To minimize it, take the derivative w.r.t. β which gives:

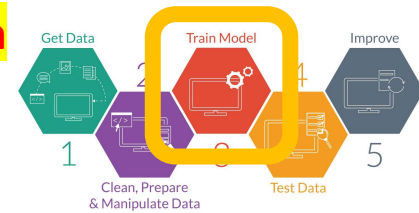
$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) = 0$$

And if $\mathbf{X}^T \mathbf{X}$ is non-singular, the unique solution is:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Iterative Regression Solutions

Optimization



Appendix (not in the exam):

- The exact method requires us to invert a matrix $(\mathbf{X}^T \mathbf{X})^{-1}$. This will often be **too big**.
- There are many **gradient-based** methods which reduce the RSS error by taking the **derivative wrt** β

$$\text{RSS}(\beta) = \sum_{i=1}^N (y_i - \beta x_i)^2$$

which was

$$\nabla = \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta)$$

Important note!

Many neural method (even in deep learning) are using gradient-based methods.

If the gradient $\rightarrow 0$ the method “gets stuck”

Gradient, gradient, gradient!

(in the exam)

- Most ML is about finding the parameters/weights → beta
- The beta is found through the gradient

$$\nabla = \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta)$$

When applying **gradient descent**, the gradient at step t :

$$\nabla^{(t)} = \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta^{(t)})$$

is used to compute the new parameter update:

$$\beta^{(t+1)} = \beta^{(t)} - \eta \cdot \nabla^{(t)}$$

gradient → update → next gradient → next update → ..

But gradient depends on....

- Gradient

$$\nabla = \mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

- The gradient at step t depends on:
 - the data $(\mathbf{X}, \mathbf{y})$
 - the current parameters $(\boldsymbol{\beta}^{(t)})$
- The parameters at step t were computed using the gradient at step $t - 1$

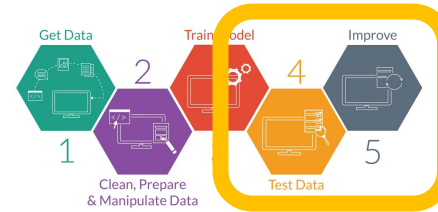
The gradient at each iteration depends not only on the data $\mathbf{X}$ and current parameters $\boldsymbol{\beta}$, but also implicitly on the **previous gradient**.

This recursive dependency drives the optimization path.

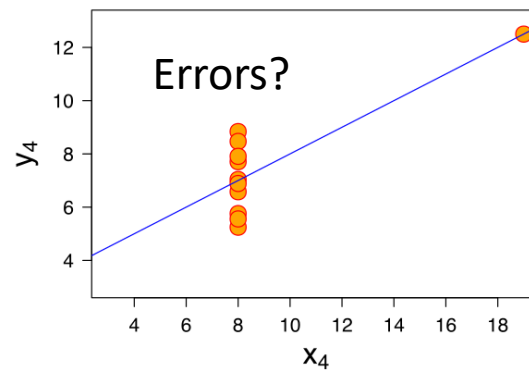
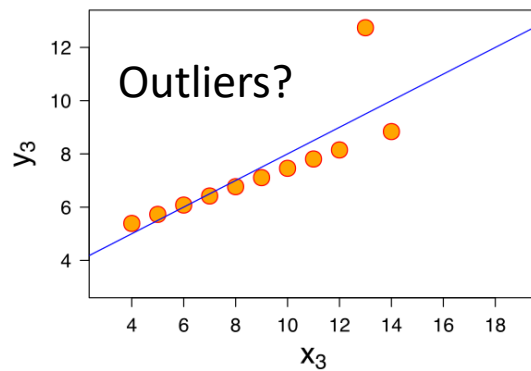
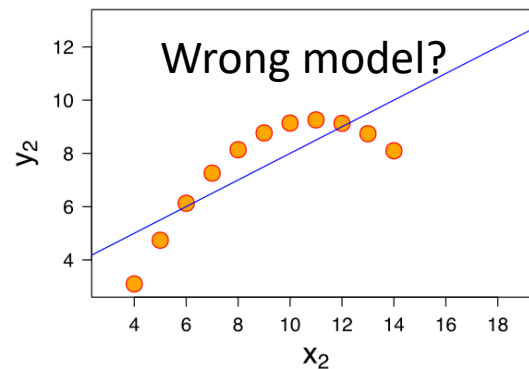
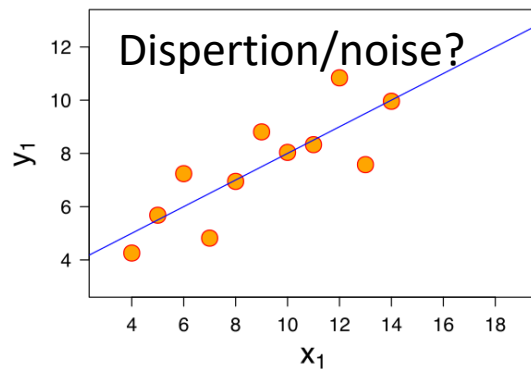
A tricky path.....

Is the regression good?

R^2 -value



- We can **always** fit a linear model to any dataset, but how do we know if there is a **real linear relationship**?



$R^2 = 0.67$ for all
Dim > 2 it is hard to
visually understand
the fitting quality!

Property	Value	Accuracy
Mean of x	9	exact
Sample variance of x : σ^2	11	exact
Mean of y	7.50	to 2 decimal places
Sample variance of y : σ^2	4.125	± 0.003
Correlation between x and y	0.816	to 3 decimal places
Linear regression line	$y = 3.00 + 0.500x$	to 2 and 3 decimal places, respectively
Coefficient of determination of the linear regression : R^2	0.67	to 2 decimal places

Anscombe's quartet: four data sets that have nearly identical simple descriptive statistics

R²-values (R-squared)

Let $\hat{y}$ be a predicted value, and $\bar{y}$ be the sample mean. Then the R-squared statistic is

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

And can be described as the fraction of the total variance not explained by the model.

R-squared

→ 1 means good fit

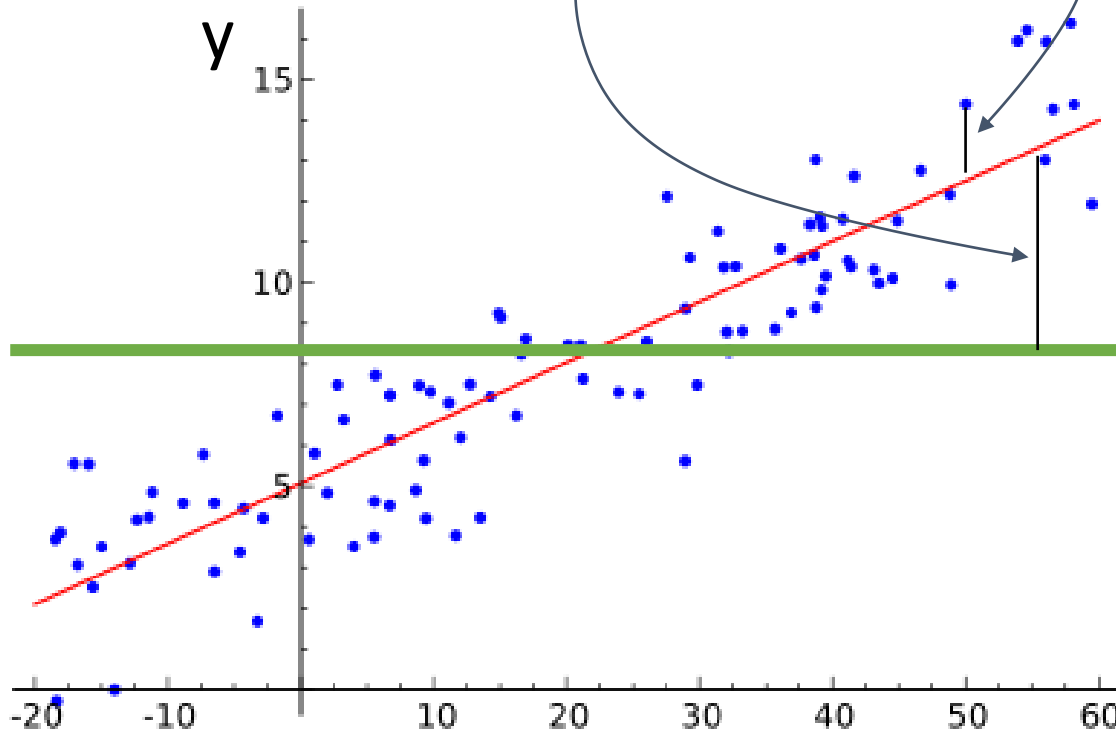
$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} =$$

sum of squares
of residuals

total sum of squares
(proportional to the
variance of the data)

Line of $\hat{y}$

Line of $\bar{y}$ (mean of data)



R^2 It must be reported!
When Dim > 2
it is quite important
(you can't see...)

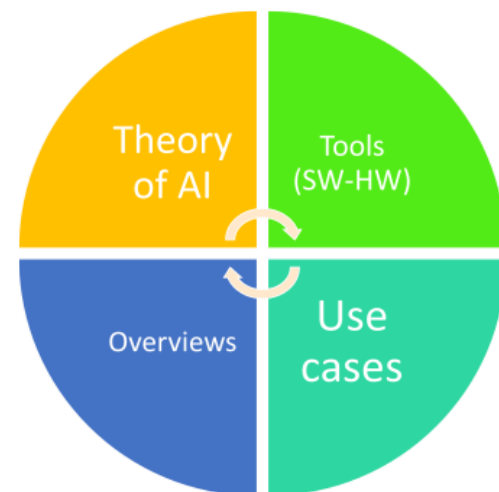


Toolboxes

Matlab and Python

Linear regressor

Linear classifiers



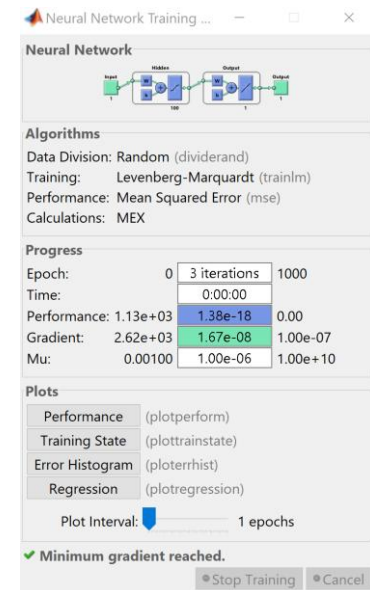
Matlab file

- 1) Follow next slides
- 2) and then lunch your scrip.



laboratory_MATLAB_linear_and_neural.m

Starting from regression
we will train and test
the first neural networks

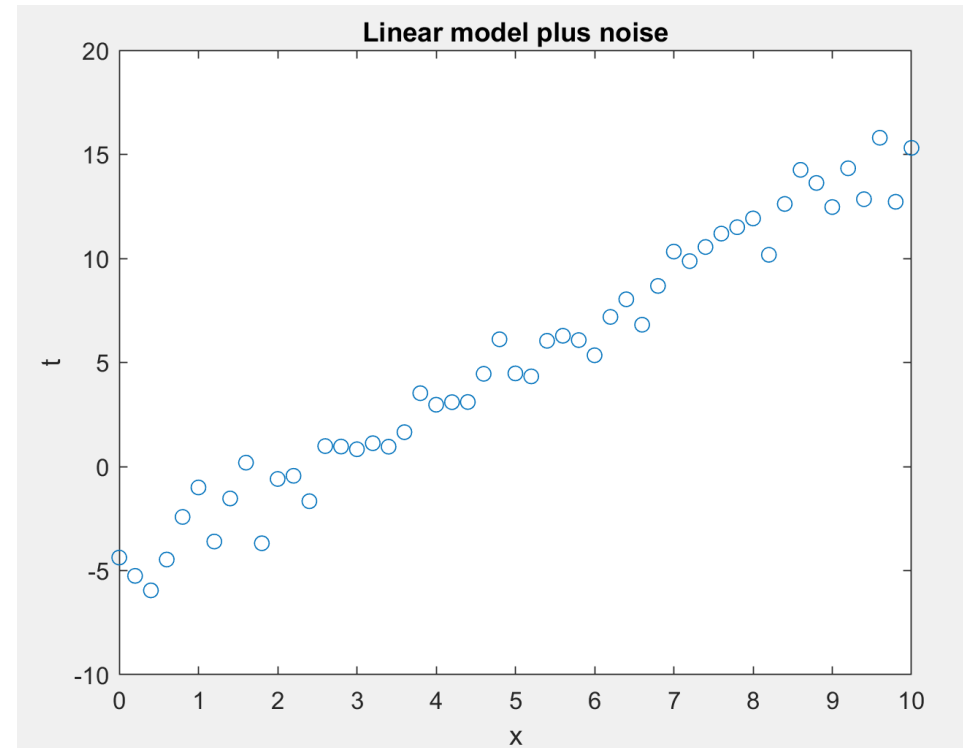


Matlab: create and train a linear regressor

Create 51 points for the eq. $t = 2x - 5$ plus gaussian noise

```
%% creating a data with known distribution
% PROBLEM 01
x = [0:0.2:10];
noise = randn(size(x));
t = 2 * x - 5 + noise;

h1 = figure ;
plot(x,t, 'o');
title('PROBLEM 01 liner data plus noise');
xlabel('x');
ylabel('t');
```



Create and train a linear regressor

net1 = newlind(x,t)

METHOD#1 (direct)

```
% create the linear model and train it  
  
if (1) % optimized solution based on the inversion of the matrix of data  
    net1 = newlind(x,t);  
else % not optimized this is the general learning for non-linear networks
```

data partition

```
end
```


Create and train a linear regressor

You can create the linear regressor using the initialization and learning for neural networks

```
%% create the linear model and train it

if (1) % optimized solution based on the inversion of the matrix of data
    % net1 = linearlayer;
else % not optimized this is the general learning for non-linear networks
    % net1 = perceptron;
    net1 = linearlayer;
    net1 = configure(net1,[0],0);

    % A little help to tune the inzial points
    net1.IW{1,1} = 3.1;
    net1.b{1} = -3.1

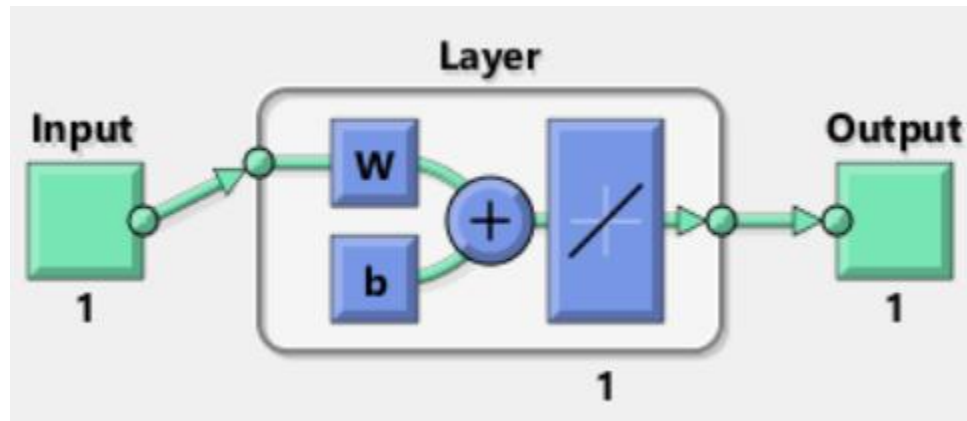
    [net1 , tr] = train(net1,x,t); % WARNING --> Train/Validation auto.created
    figure(h2); plotperform(tr) % need more tuning..... :-(
end
```

METHOD#2

- inzialization as linear net
- 1 input 1 layer
- Weight inzialization
- Levemberg-Marquardt optimization
- (general, but not optimized)

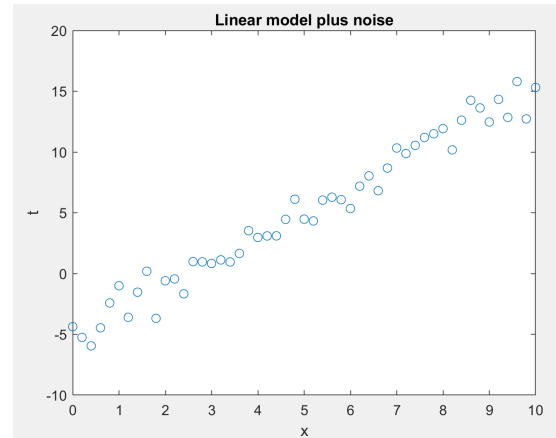
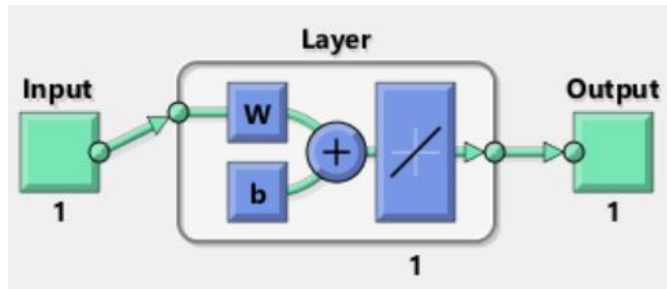
The created linear regressor

```
%% visualization of the trained linear model  
view(net1)
```



$$\text{Output} = w \text{ input} + b$$

Access the parameters



```
% how to access the weight of the model
```

```
W = net1.IW{1,1} ;
```

```
b = net1.b{1} ;
```

```
fprintf('-----\n');
```

```
fprintf('trained linear model W = %f \n' , W);
```

```
fprintf('trained linear model b = %f \n' , b);
```

```
fprintf('-----\n');
```

```
x = [0:0.2:10];  
noise = randn(size(x));  
t = 2 * x - 5 + noise;
```

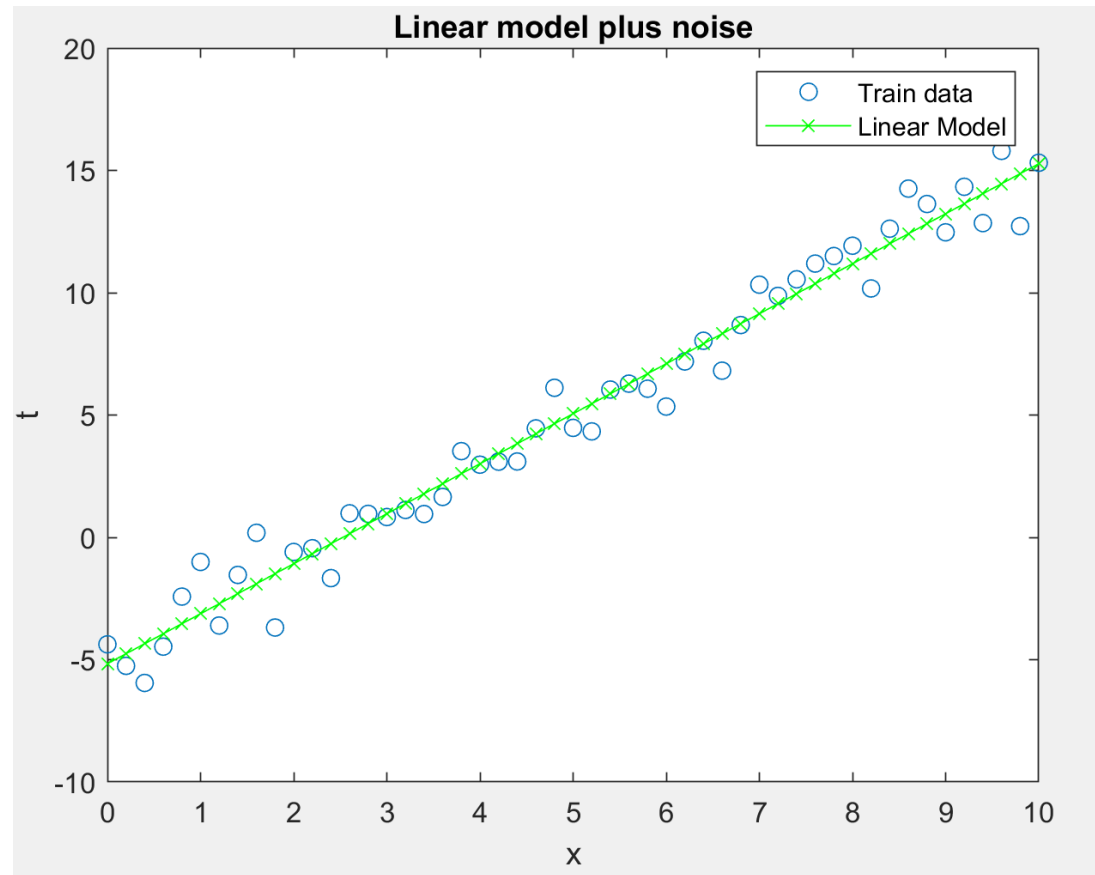
```
-----  
trained linear model W = 2.043733  
trained linear model b = -5.165622  
-----
```

Processing inputs and plotting the results

```
h1 = figure ;  
plot(x,t, 'o');  
title('Linear model plus noise');  
xlabel('x');  
ylabel('t');  
  
% how to check the performance  
y1 = net1(x); % simulate the inputs  
perf = perform(net1,y1,t) % Error on **trainig**  
  
% plot the output of the model  
figure(h1)  
hold on  
plot(x, y1, 'xg-')  
legend('Train data', 'Linear Model')
```

net1 = newlind(x,t)

y1 = net1(x)



Accuracy assessment

%% how to check the performance

```
y1 = net1(x); % simulate the inputs
perf = perform(net1,y1,t) % Error on **trainig**
net1.performFcn % is telling you the error metrics (MSE for Lin.)
mse_check = sum((y1 - t).^2 )/ size(t,2);
mean_t = mean(t);
R2 = 1 - ( sum((t - y1).^2 )/sum((t - mean_t).^2) );
fprintf('-----trained linear model-----\n');
fprintf('Perf.           = %f \n' , perf );
fprintf('Type of index = %s \n' , net1.performFcn );
fprintf('MSE             = %f \n' , mse_check );
fprintf('R2              = %f \n' , R2 );
fprintf('-----\n');
```

$$MSE = \sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

****WARNING****

This is about TRAIN dataset
You need to re-do the measures
on a TEST dataset

```
-----trained linear model-----
Perf.           = 0.836647
Type of index = mse
MSE             = 0.836647
R2              = 0.976989
-----
```

Try a feedforward neural network!

```
net2 = feedforwardnet (3)
```

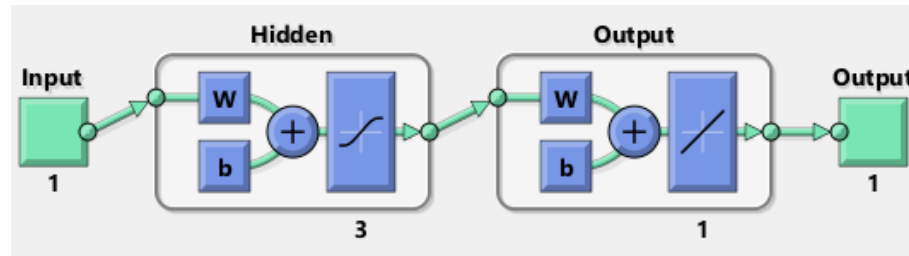
Initialization with 3 neurons

```
net2 = train(net2,x,t);
```

training

```
y2 = net2(x)
```

running



Try a feedforward neural network!

```
%% % Let's face the OCCAM's RAZOR --> use a large network
net2 = feedforwardnet(3);

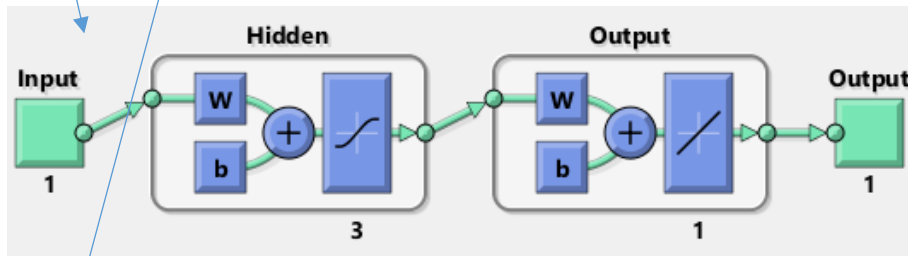
net2.divideParam.trainRatio = 1.0;
net2.divideParam.testRatio = 0.0;
net2.divideParam.valRatio = 0;

[net2 , tr] = train(net2,x,t);

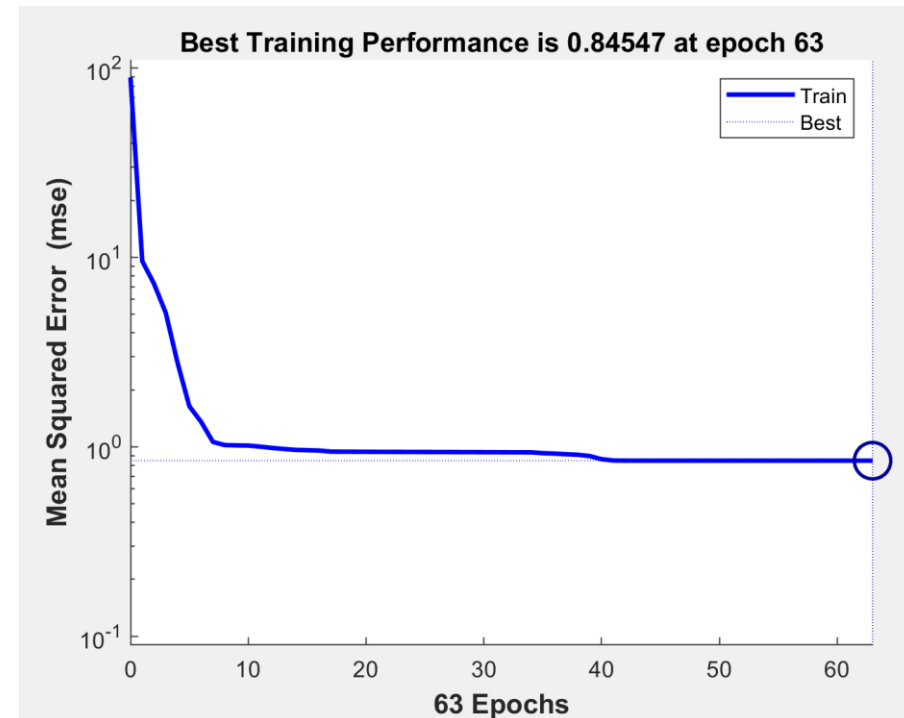
h2 = figure;
plotperform(tr)

view(net2)
y2 = net2(x);
perf = perform(net2,y2,t) % Error on **trainig**
```

Force to use
all data in
the TRAIN set



perf =
0.8455



MSE

Algorithms

Data Division: Random (dividerand)
Training: Levenberg-Marquardt (trainlm)
Performance: Mean Squared Error (mse)
Calculations: MEX

Progress

Epoch:	0	63 iterations	1000
Time:	0:00:00		
Performance:	89.2	0.845	0.00
Gradient:	179	8.16e-08	1.00e-07
Mu:	0.00100	1.00e-20	1.00e+10

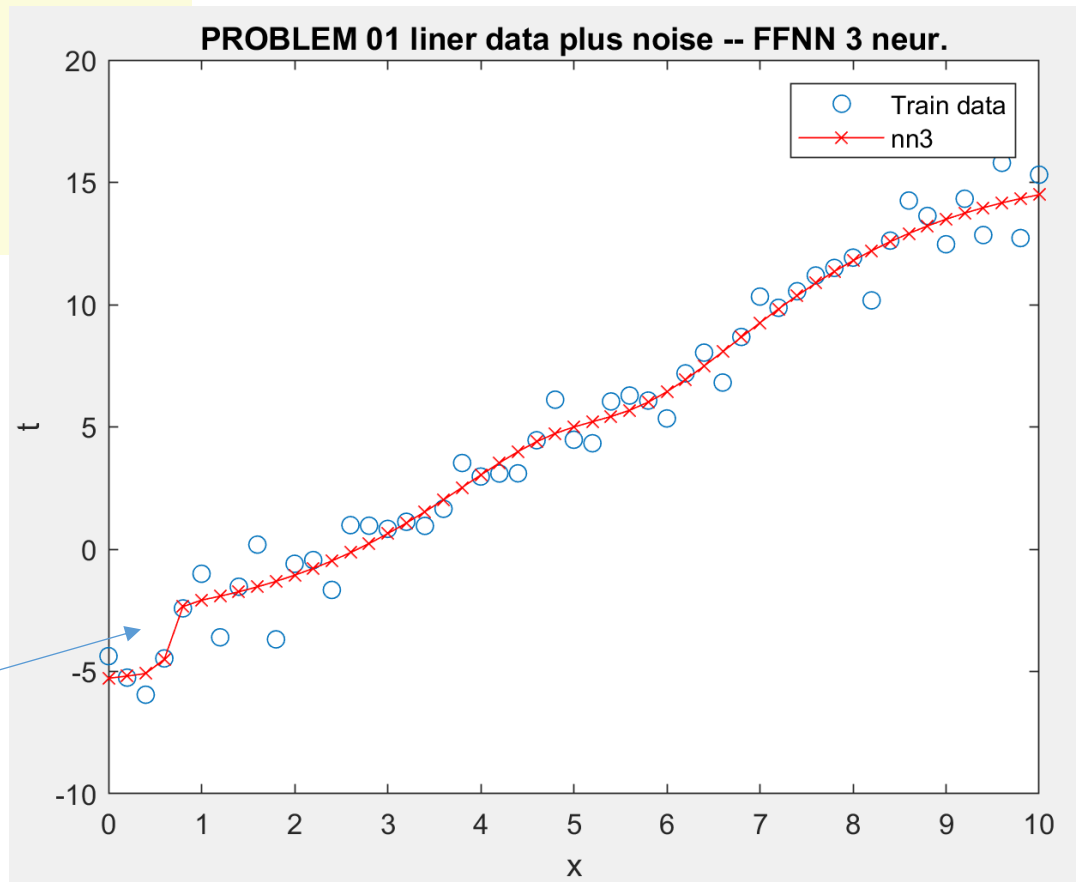
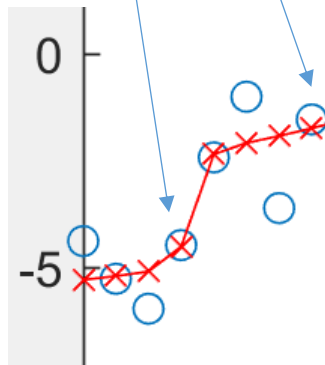
Try a feedforward neural network!

```
h3 = figure
plot(x,t, 'o');
title('PROBLEM 01 liner data plus noise -- FFNN 3 neur. ');
xlabel('x');
ylabel('t');

hold on
plot(x,y2, 'xr-');
xlabel('x');
ylabel('t');
legend('Train data', 'nn3')
```

Noise
fitting?

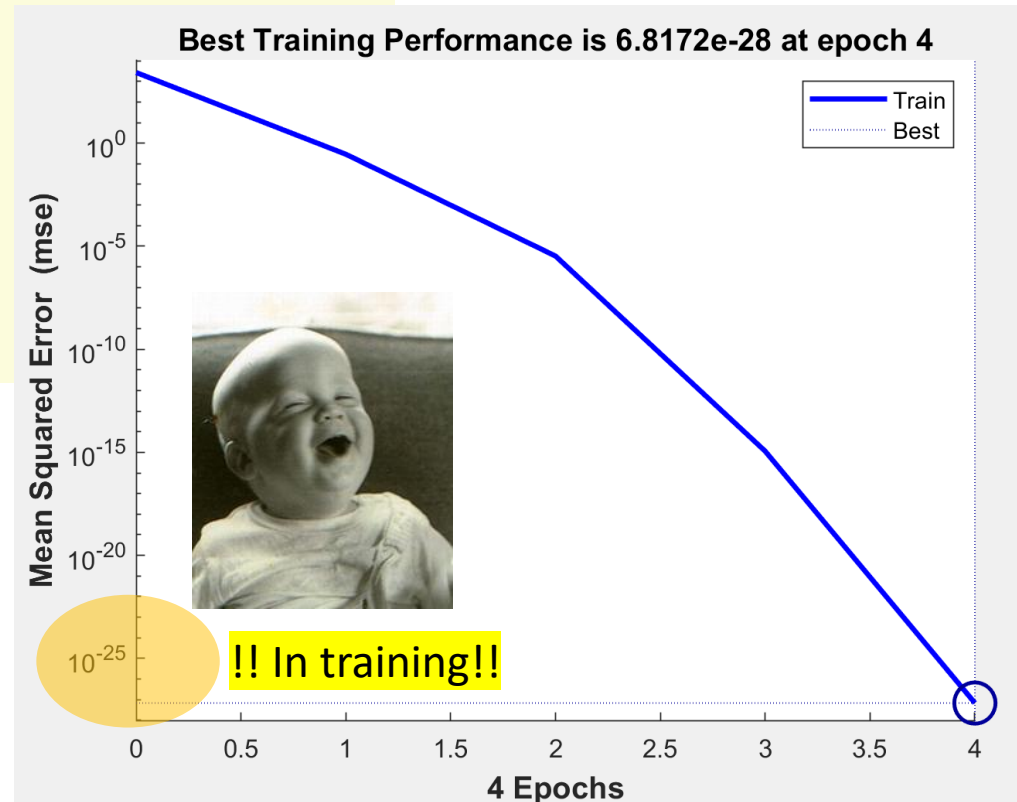
correct



Wrong design: 100 neurons with 51 data points!

```
%% real overfitting
% Let's face the OCCAM's RAZOR --> use very large network
net3 = feedforwardnet(100);
net3.divideParam.trainRatio = 1.0;
net3.divideParam.testRatio  = 0.0;
net3.divideParam.valRatio   = 0;

[net3 , tr] = train(net3,x,t);
figure(h2); plotperform(tr)
view(net3)
y3 = net3(x);
```

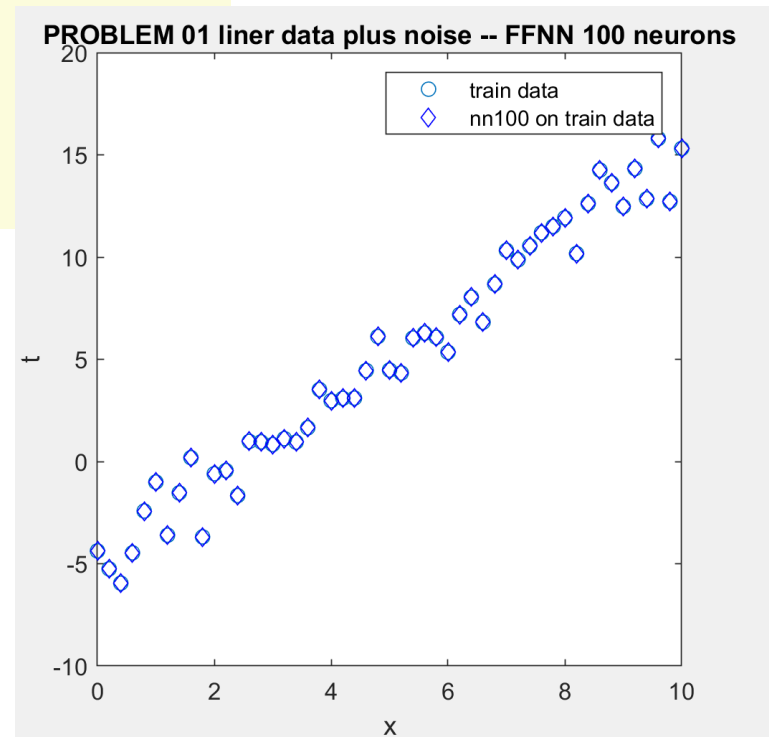
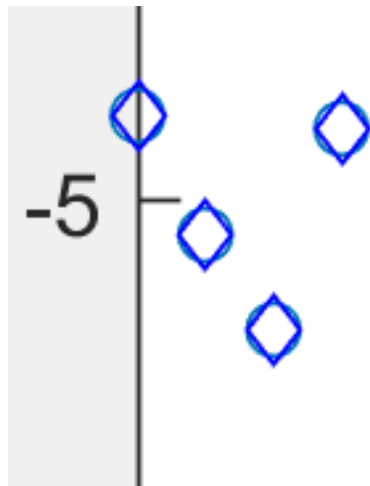


Wrong design: 100 neurons with 51 data points!

```
h4 = figure
subplot(1,2,1)
plot(x,t, 'o');
title('PROBLEM 01 liner data plus noise -- FFNN 100 neurons');
xlabel('x');
ylabel('t');
hold on
plot(x,y3, 'db');
legend('train data', 'nn100 on train data')
```



○ train data
◇ nn100 on train data



Let's built a test set....

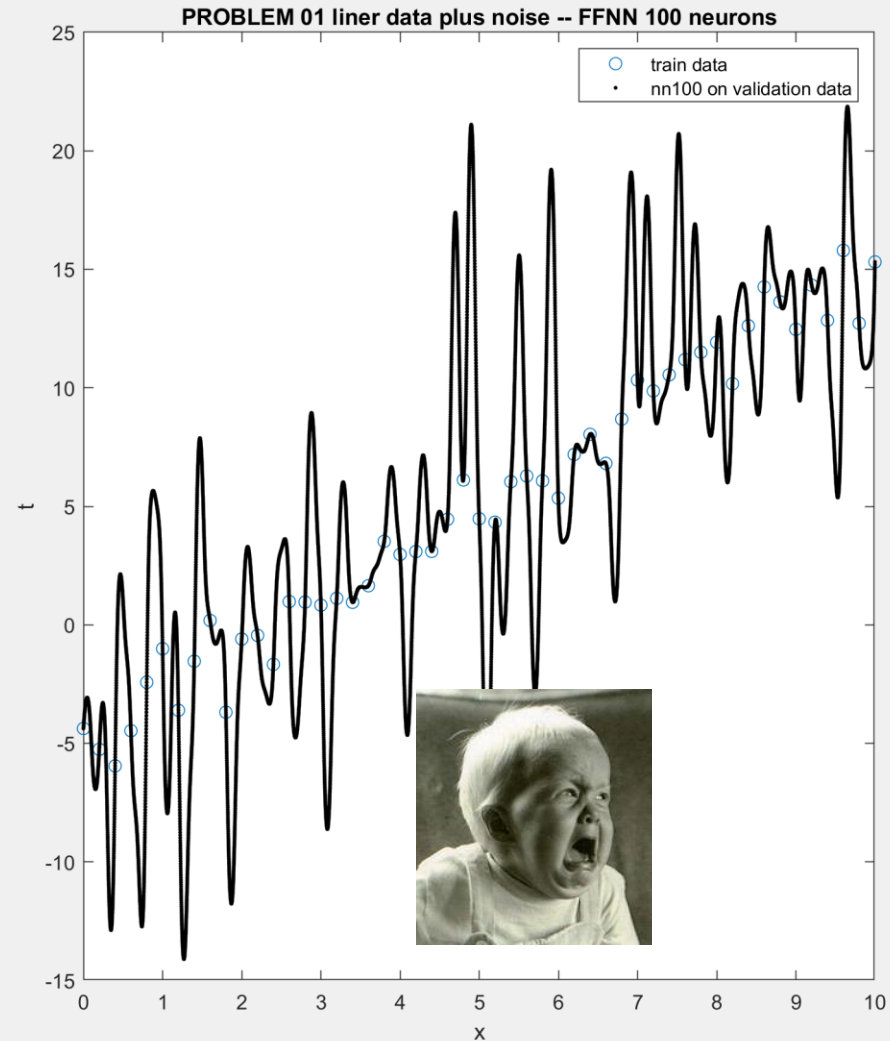
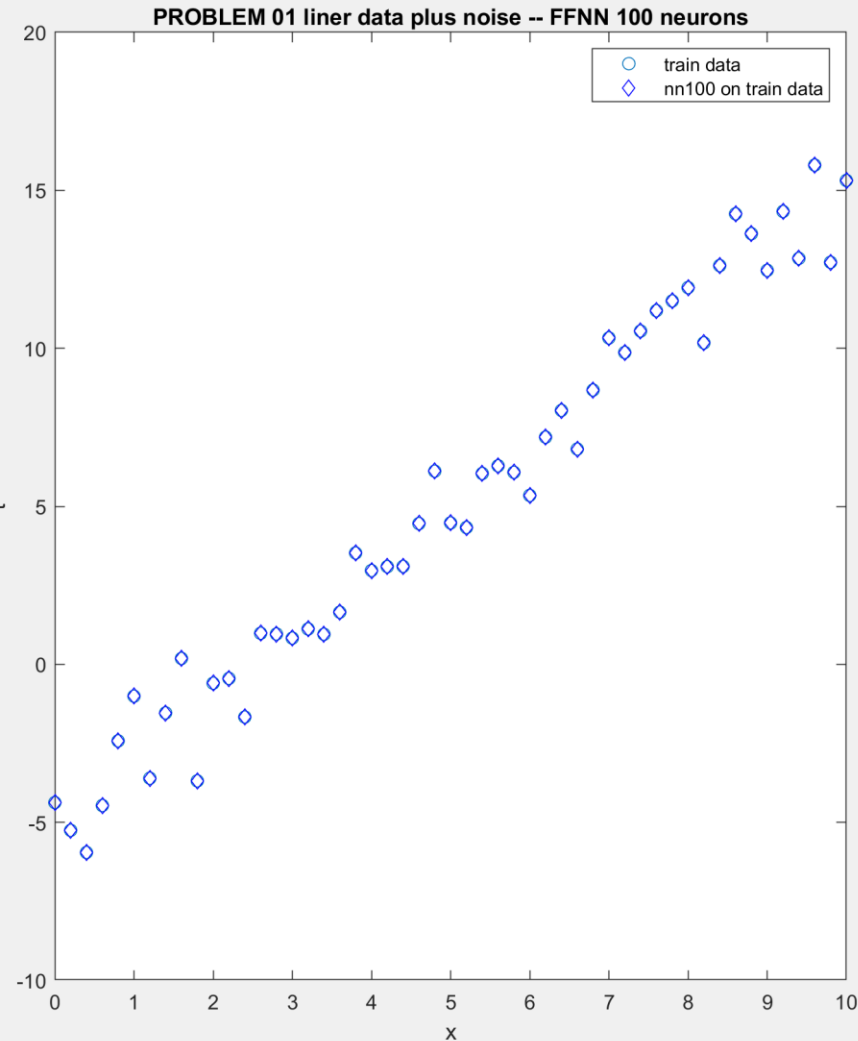
Let's inject in the net3 (100 neurons) more x points...

```
%% test point
x_bis = [0:0.0005:10];
y3_bis = net3(x_bis);

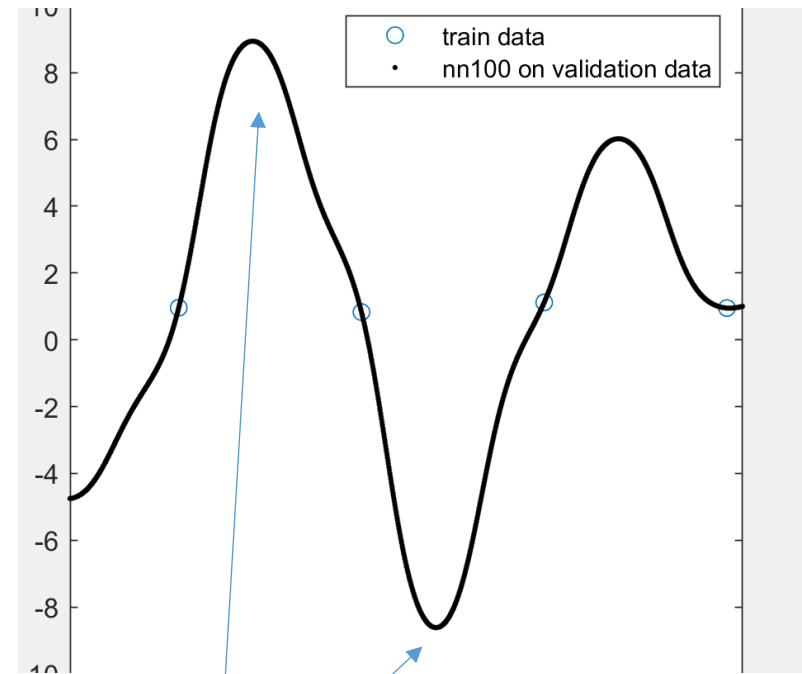
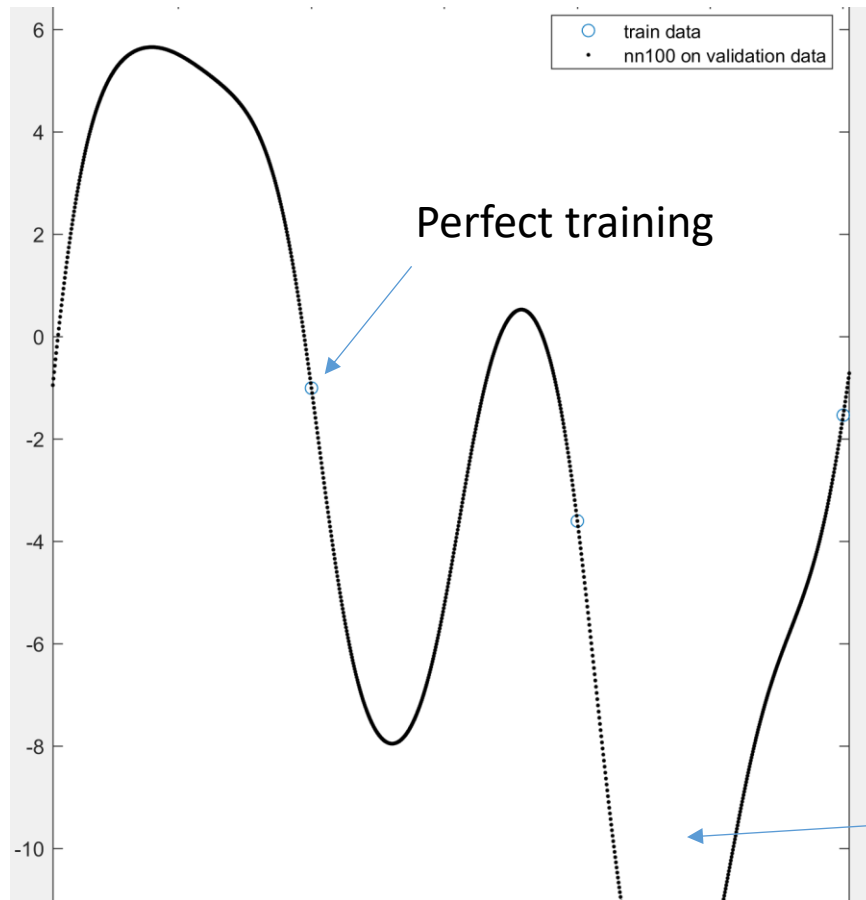
perf = perform(net3,y3,t) % Error on **trainig**

subplot(1,2,2)
plot(x,t, 'o');
title('PROBLEM 01 liner data plus noise -- FFNN 100 neurons');
xlabel('x');
ylabel('t');
hold on
plot(x_bis,y3_bis, '.k');
legend('train data', 'nn100 on validation data' )
```

...and we overfitted (a lot!)



Remember next time you will initialize a model with **#DoF >> #samples...**



Very bad
generalization



Python file in Colab

- 1) Follow next slides
- 2) and then lunch your script.

`laboratory_COLAB_linear_regressors.ipynb`

Let's review how to
create a regressor,
check accuracy and
doing some plots

Import the needed libraries:

- plotting
- numerical processing

```
import matplotlib.pyplot as plt  
import numpy as np
```



It adds support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.

Data preparation

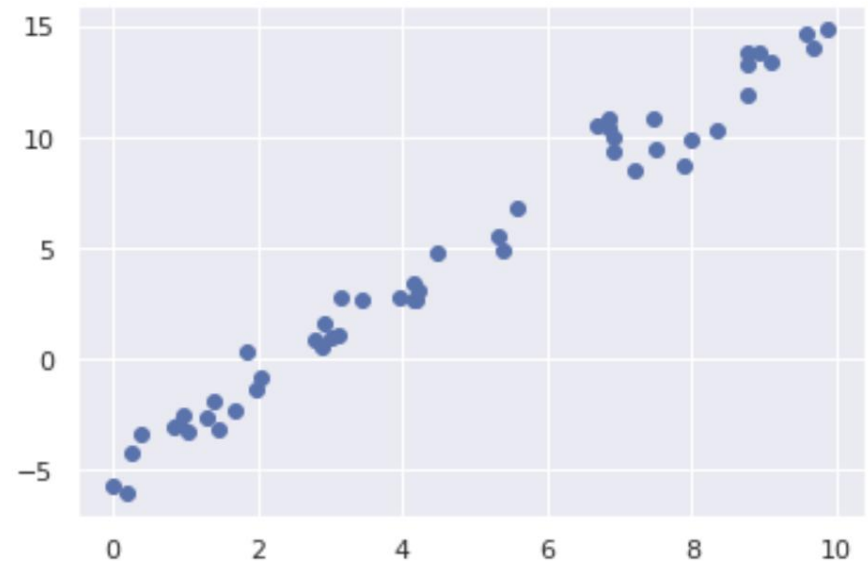
1D Linear Regression

We want to create a model which is a straight-line fit of the form

$$y = ax + b$$

Consider the following data, which is scattered about a line with a slope of 2 and an intercept of -5:

```
[35] rng = np.random.RandomState(1)
      num_points = 50
      x = 10 * rng.rand(num_points)
      t = 2 * x - 5 + rng.randn(num_points)
      plt.scatter(x, t);
```



Creation of the linear regressor

We can use **Scikit-Learn**'s LinearRegression estimator to fit this data and construct the best-fit line:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)

model.fit(x[:, np.newaxis], t) # newaxis adds a dimension (needed from the function)

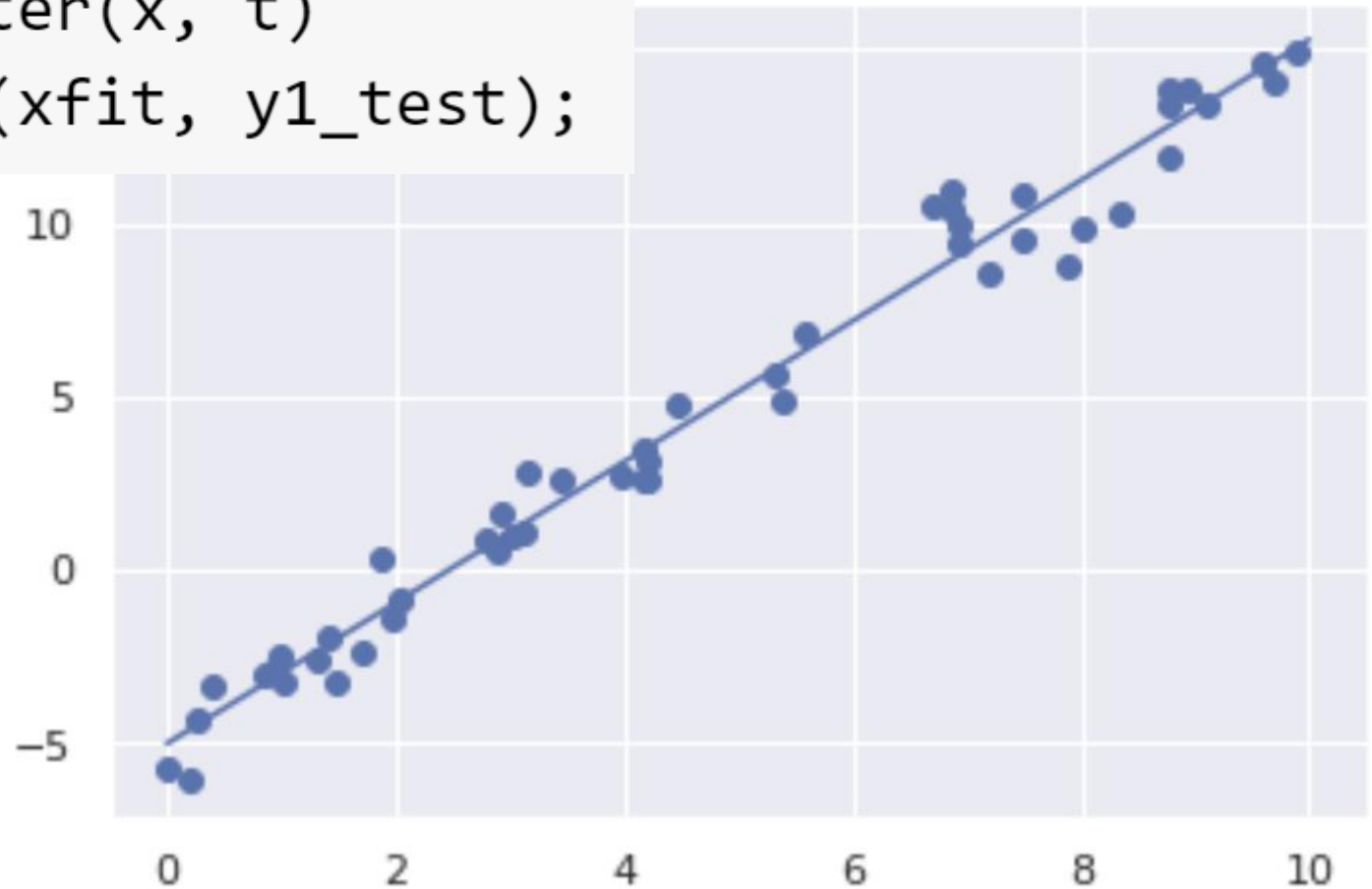
y1 = model.predict(x[:, np.newaxis]) # let's create the predicted values

xfit = np.linspace(0, 10, 1000) # let's create an array to test the regressor [0 ... 10] of 1000 points
y1_test = model.predict(xfit[:, np.newaxis]) # let's create the predicted values
```

```
model = LinearRegression(fit_intercept=True)
model.fit(x[:, np.newaxis], t)
y1 = model.predict(x[:, np.newaxis])
```

Plotting

```
plt.scatter(x, t)  
plt.plot(xfit, y1_test);
```



Get the parameters of the model

```
[37] print("Model slope:      ", model.coef_[0])  
      print("Model intercept:", model.intercept_)
```

```
↳ Model slope:      2.0272088103606953  
   Model intercept: -4.998577085553204
```

Accuracy assessement

```
[38] from sklearn.metrics import mean_squared_error
      from math import sqrt

      # Mean squared error
      mse = mean_squared_error(y1, t)
      rmse = sqrt(mse)

      #R^2
      from sklearn.metrics import r2_score
      R2 = r2_score(y1, t)

      print("MSE:      ", mse )
      print("RMSE:     ", rmse )
      print("R2:       ", R2  )
```

```
☞ MSE:      0.8183388570266171
   RMSE:     0.9046208360559783
   R2:       0.9786330659856474
```

**** WARNING ****
You have to create a TEST
dataset to get the real
accuracy in generalization.
This is processed on the
TRAIN dataset.

You will find different numbers w.r.t. Matlab
due to the different data initialization

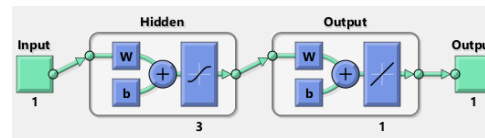
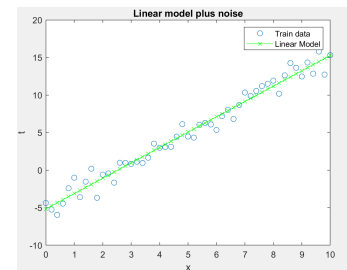
Main points



- Agents Memory and Context management
- Loading, EDA/plotting in Matlab
- New model? What to know? What to do?
- The Bias/Variance Tradeoff (Dilemma)
- Creation and use (with code) of the first complete machine learning model
 - Linear regressors
 - First neural network
 - Accuracy assessment

```
net1 = newlind(x,t)
```

```
y1 = net1(x)
```



```
net2 = feedforwardnet (3)
```

```
net2 = train(net2,x,t);
```

```
y2 = net2(x)
```